ONTWIKKELDOCUMENT · LEERUITKOMST 1

Rent my car

Ontwerp, realisatie en testen van een Web API met Ktor

Module	AI-powered software development
Opleiding	Informatica, jaar 2 (Avans Hogeschool)
Student	Thimo Harte
Versie	25 oktober 2026
Repository	github.com/ThimoHarte/RentMyCar

Inhoud

0. Inleiding 3
- 0.1 Aanleiding en doel 3
- 0.2 De casus: Rent my car 3
- 0.3 Afbakening 3
- 0.4 Technische opzet en projectstructuur 4
- 0.5 Leeswijzer 5

1. Analyse 5
- 1.1 Functionele eisen 6
- 1.2 Niet-functionele eisen 7
- 1.3 Actoren en rollen 8
- 1.4 Gegevens 8
- 1.5 User stories 10
- 1.6 Permissiematrix 10
- 1.7 Use cases 11
- 1.8 Use case diagram 14

2. Ontwerp 15
- 2.1 Domeinmodel (klassendiagram) 16
- 2.2 Toelichting op de OO-keuzes 18
- 2.3 Packagediagram 21
- 2.4 Entiteit-relatiediagram (ERD) 23
- 2.5 Relationeel databasemodel 25

3. Web API 29
- 3.1 Architectuur 29
- 3.2 Endpoints 32
- 3.3 Foutafhandeling en statuscodes 37
- 3.4 Authenticatie en autorisatie 37
- 3.5 AI-agent 41
- 3.6 Sequence diagrams 41

4. Testen 47
- 4.1 Teststrategie 47
- 4.2 Unit tests 49
- 4.3 Integratietests 50
- 4.4 Testrapport API-requests 51

5. Installatie en gebruik 54
- 5.1 Vereisten 54
- 5.2 Project starten 54
- 5.3 Testdata en testaccounts 55
- 5.4 De API gebruiken 55

6. Productverantwoording 57
- 6.1 Verantwoording AI-gebruik 57
- 6.2 Highlights 58
- 6.3 Lowlights en verbeterpunten 58
- 6.4 Lessons learned en advies 58

7. Android app 60

Bijlage A – Aannames 61

0. Inleiding

0.1 Aanleiding en doel

Dit document hoort bij de module *AI-powered software development* van de opleiding Informatica van Avans Hogeschool (jaar 2). Het beschrijft de uitwerking van leeruitkomst 1: *AI-powered backend development*.

Het doel van dit document is het ontwerp, de realisatie en het testen te beschrijven van een Web API voor de casus Rent my car. De Web API is gebouwd in Kotlin met het framework Ktor. In periode 2 wordt deze API gebruikt door een Android-app.

Ik werk dit project alleen uit. Dit is afgestemd met de docent.

0.2 De casus: Rent my car

Rent my car is een platform voor het delen van auto's. Het doel is duurzamer vervoer: een auto die stilstaat, kan door iemand anders worden gebruikt.

Gebruikers kunnen op het platform:

- hun eigen auto aanbieden om te verhuren aan andere gebruikers;
- een auto van een ander zoeken, reserveren en huren.

Het platform kent drie soorten auto's:

- **ICE** (Internal Combustion Engine): een auto met een verbrandingsmotor op benzine, diesel of LPG;
- **BEV** (Battery Electric Vehicle): een elektrische auto met een accu;
- **FCEV** (Fuel Cell Electric Vehicle): een elektrische auto die stroom maakt uit waterstof.

Een eigenaar kan per auto de kosten per kilometer en de TCO (Total Cost of Ownership, de totale kosten per jaar) laten berekenen. Die berekening verschilt per soort auto.

Een huurder kan auto's zoeken en filteren, bijvoorbeeld op afstand of prijs, en een auto reserveren voor een bepaalde periode. Daarna kan de huurder de route naar de auto opvragen. Tijdens het rijden houdt de app het rijgedrag bij; rustig rijden levert bonuspunten op.

0.3 Afbakening

De casus is groot. Daarom maak ik onderscheid tussen wat in de backend (periode 1) komt en wat pas in de app (periode 2) wordt uitgewerkt.

Wel in de backend (periode 1):

- auto's beheren: toevoegen, bekijken, zoeken en filteren (op soort, plaats en maximale uurprijs), wijzigen en verwijderen;
- de kosten per kilometer en de TCO berekenen;
- gebruikers registreren en laten inloggen;
- reserveringen aanvragen, die de eigenaar goedkeurt of afwijst, met controle dat een auto niet twee keer voor dezelfde periode wordt verhuurd;
- een AI-agent die een deel van de functionaliteit uitvoert (de precieze invulling volgt; het taalmodel draait lokaal en is gratis). *Nog niet gerealiseerd.*

Pas in periode 2:

- foto's van een auto uploaden, bekijken en verwijderen. Dit stond gepland voor periode 1, maar is nog niet gerealiseerd;
- auto's zoeken op beschikbaarheid in een periode. Ook dit is nog niet gerealiseerd;
- de plek van de auto op een kaart aanwijzen en auto's tonen op een kaart;
- zoeken op afstand (hiervoor zijn coördinaten nodig, die in periode 2 via de kaart worden vastgelegd);
- de looproute naar de auto;
- het meten van rijgedrag met de sensoren van de telefoon en het toekennen van bonuspunten.

De backend krijgt voor deze onderdelen alleen wat nodig is om de gegevens op te slaan en op te vragen.

Buiten scope:

- echte betalingen;
- een digitale controle van identiteit of rijbewijs. De eigenaar controleert het rijbewijs zelf bij de sleuteloverdracht (zie hoofdstuk 1).

0.4 Technische opzet en projectstructuur

Het project is geschreven in **Kotlin**. Als build tool gebruik ik de **Kotlin Toolchain** in plaats van Gradle, zoals aanbevolen in de module.

Het project is aangemaakt met de Kotlin Multiplatform-wizard (kmp.jetbrains.com), met als doelen Android en Server. Daardoor is de structuur meteen geschikt voor periode 2.

Module	Doel	Periode
`server`	De Web API, gebouwd met Ktor	1
`core`	Gedeelde code voor server en app: de DTO's (de gegevens die via de API worden verstuurd) en de enums	1 en 2
`app/shared`	De gebruikersinterface (Compose) en de logica van de app	2
`app/androidApp`	De Android-app zelf	2

De packages in de code beginnen met `rentmycar`, bijvoorbeeld `rentmycar.server.cars` en `rentmycar.core` (zie hoofdstuk 2.3).

Ik heb voor deze opzet gekozen omdat de server en de app dezelfde datamodellen uit `core` gebruiken. Als ik een veld toevoeg of wijzig, geldt dat direct voor beide kanten. Dat voorkomt verschillen tussen wat de API verstuurt en wat de app verwacht.

Opslag van gegevens: de gegevens worden opgeslagen in een **H2-database**. H2 draait mee in de applicatie, waardoor er geen aparte databaseserver geïnstalleerd hoeft te worden. Voor de toegang tot de database gebruik ik de Kotlin-library **Exposed**. De rest van de applicatie praat met de database via een *repository-interface*. Daardoor kan ik tijdens het bouwen en testen eerst met gegevens in het geheugen werken en daarna overstappen op de database, zonder dat de rest van de code hoeft te veranderen.

0.5 Leeswijzer

Hoofdstuk	Inhoud
1. Analyse	Eisen, rollen, user stories, permissiematrix en use cases
2. Ontwerp	Klassendiagram, toelichting op de OO-keuzes, packagediagram en databasemodel
3. Web API	Architectuur, endpoints, foutafhandeling, authenticatie, de AI-agent en sequence diagrams
4. Testen	Teststrategie, unit tests, integratietests en testrapport
5. Installatie en gebruik	Hoe de applicatie wordt gestart en getest, inclusief testdata
6. Productverantwoording	Verantwoording van het AI-gebruik, highlights, lowlights en lessons learned
7. Android app	Uitwerking in periode 2

Daarnaast zijn er twee losse documenten:

- `aannames.md`: de vaste waarden die in de berekeningen worden gebruikt, zoals energieprijzen;
- het **AI-log** (los document, bron `ai-log.md`): hoe ik AI heb gebruikt tijdens het project.

Taalafspraken: dit document en de commitberichten zijn in het Nederlands. De code, inclusief de namen en het commentaar, is in het Engels, omdat de opdracht Engelstalige broncode vraagt.

1. Analyse

In dit hoofdstuk staat wat de applicatie moet kunnen. De eisen komen uit de casus Rent my car en uit de minimale eisen van de opdracht. Waar de casus niets zegt, heb ik zelf een keuze gemaakt; die keuzes staan erbij.

1.1 Functionele eisen

De kolom *Status* geeft aan of een eis in de huidige versie van de software zit: ✅ gerealiseerd, ➡️ SOON volgt in periode 2, ⏳ nog niet gerealiseerd.

Nr	Eis	Status
FE-01	Een bezoeker kan een account aanmaken.	✅
FE-02	Een gebruiker kan inloggen met e-mailadres en wachtwoord.	✅
FE-03	Een gebruiker kan een auto aanbieden met gegevens, adres en tarieven.	✅
FE-04	Er zijn drie soorten auto's: ICE (brandstof), BEV (elektrisch met accu) en FCEV (waterstof). Ze hebben gedeelde gegevens en gedeelde functionaliteit.	✅
FE-05	De eigenaar kan zijn auto wijzigen.	✅
FE-06	De eigenaar kan zijn auto verwijderen, zolang er geen aangevraagde of goedgekeurde reserveringen in de toekomst zijn. De auto wordt niet echt verwijderd, maar op "niet actief" gezet, zodat oude reserveringen bewaard blijven.	✅
FE-07	De eigenaar kan foto's toevoegen aan zijn auto en weer verwijderen.	➡️ SOON Periode 2
FE-08	De eigenaar kan de kosten per kilometer van zijn auto laten berekenen. De berekening verschilt per soort auto.	✅
FE-09	De eigenaar kan de TCO (totale kosten per jaar) van zijn auto laten berekenen, uitgaande van 15.000 km per jaar.	✅
FE-10	Iedereen kan een lijst met auto's bekijken en zoeken en filteren op soort auto, plaats, maximale uurprijs en beschikbaarheid in een periode.	✅ Deels: filteren op beschikbaarheid volgt in periode 2
FE-11	Iedereen kan de details van een auto bekijken, inclusief foto's.	✅ Deels: foto's volgen in periode 2
FE-12	Een gebruiker kan een reservering aanvragen voor een auto van een ander, met een start- en eindmoment en het verwachte aantal kilometers.	✅
FE-13	Bij een aanvraag ziet de huurder de geschatte huurprijs: uurtarief × aantal uren + km-tarief × verwachte kilometers. Deze prijs wordt bij het aanvragen vastgelegd en verandert niet als de eigenaar later zijn tarieven aanpast.	✅
FE-14	De eigenaar kan een aanvraag goedkeuren of afwijzen.	✅
FE-15	Een aanvraag kan alleen worden goedgekeurd als de auto in die periode niet al is verhuurd (geen overlap met een andere goedgekeurde reservering).	✅
FE-16	De huurder kan een reservering annuleren zolang de huurperiode nog niet is begonnen.	✅

Nr	Eis	Status
FE-17	Een gebruiker kan zijn eigen reserveringen bekijken.	✅
FE-18	De eigenaar kan de aanvragen en reserveringen voor zijn auto's bekijken.	✅
FE-19	Na goedkeuring ziet de huurder de contactgegevens van de eigenaar, zodat zij de sleuteloverdracht kunnen afspreken.	✅
FE-20	Een AI-agent voert een afgebakend deel van de functionaliteit uit (invulling volgt in hoofdstuk 3).	⏳ Nog niet gerealiseerd

Regels voor een reservering:

- het eindmoment ligt na het startmoment;
- een reservering duurt minimaal 1 uur;
- het startmoment ligt in de toekomst;
- een gebruiker kan zijn eigen auto niet reserveren;
- het verwachte aantal kilometers is niet negatief.

1.2 Niet-functionele eisen

Nr	Eis	Bron
NFE-01	De Web API is geschreven in Kotlin met het framework Ktor.	Opdracht
NFE-02	Gegevens worden opgeslagen in een database (H2 met Exposed). Bij de start bevat de database realistische testdata.	Opdracht + eigen keuze
NFE-03	De API gebruikt JSON en geeft passende HTTP-statuscodes terug, ook bij fouten.	Opdracht (LU1)
NFE-04	Acties die een account vereisen, kunnen alleen met een geldig inlogtoken (JWT) worden uitgevoerd. Een token is 15 minuten geldig.	Eigen eis, OWASP en gastdocent
NFE-05	Wachtwoorden worden niet leesbaar opgeslagen, maar gehasht met Argon2id.	Eigen eis, OWASP en gastdocent
NFE-06	De broncode is in het Engels en volgt de Kotlin-conventies voor namen. Package-namen passen bij de architectuur.	Opdracht
NFE-07	Er zijn minstens 3 unit tests en 3 integratietests, en tests voor de vier soorten CRUD-requests.	Opdracht
NFE-08	De applicatie draait lokaal op mijn eigen computer.	Opdracht
NFE-09	Het taalmodel van de AI-agent draait lokaal en is gratis.	Eigen eis
NFE-10	Inloggen en registreren zijn beperkt tot 10 pogingen per minuut, tegen het raden van wachtwoorden.	OWASP, docent en gastdocent

1.3 Actoren en rollen

Actor	Omschrijving
Bezoeker	Iemand die niet is ingelogd. Kan auto's zoeken en bekijken en een account aanmaken.
Gebruiker	Iemand die is ingelogd. Kan auto's aanbieden en huren.
Eigenaar	Een gebruiker in de rol van aanbieder van een auto. Een gebruiker is eigenaar van de auto's die hij zelf heeft toegevoegd.
Huurder	Een gebruiker in de rol van iemand die een auto reserveert.

Eigenaar en huurder zijn geen aparte accounts. Eén gebruiker kan beide rollen hebben, net als bij platforms zoals SnappCar.

Keuze: er is geen beheerder (admin). Dit houdt de applicatie eenvoudig en is voor de casus niet nodig.

1.4 Gegevens

Gebruiker

- naam, e-mailadres, wachtwoord (gehasht)
- bonuspunten (worden in periode 2 gebruikt voor het belonen van rustig rijgedrag)

Auto (voor alle soorten)

- merk, model, bouwjaar, kenteken, aantal zitplaatsen
- uurtarief en km-tarief
- adres: straat, huisnummer, postcode, plaats
- aanschafprijs (nodig voor de TCO)
- actief of niet actief (een verwijderde auto wordt op "niet actief" gezet)
- eigenaar
- foto's (volgt in periode 2)

Extra per soort auto

Soort	Extra gegevens
ICE	brandstofsoort (benzine, diesel of LPG) en verbruik in liter per 100 km
BEV	verbruik in kWh per 100 km
FCEV	verbruik in kg waterstof per 100 km

Reservering

- auto en huurder
- startmoment en eindmoment
- verwacht aantal kilometers
- status: `AANGEVRAAGD`, `GOEDGEKEURD`, `AFGEWEZEN` of `GEANNULEERD`
- geschatte huurprijs (wordt bij het aanvragen berekend en vastgelegd)

De vaste getallen die nodig zijn voor de berekeningen, zoals energieprijzen en de bedragen voor de TCO, staan in `aannames.md`.

Levenscyclus van een reservering



- Alleen een aanvraag (`AANGEVRAAGD`) kan worden goedgekeurd of afgewezen.
- Goedkeuren wordt geweigerd als er al een goedgekeurde reservering voor dezelfde auto is die overlapt.
- Annuleren kan bij `AANGEVRAAGD` en `GOEDGEKEURD`, zolang het startmoment nog niet is bereikt.
- `AFGEWEZEN` en `GEANNULEERD` zijn eindtoestanden.

Verwijderen van een auto

Een auto wordt niet echt uit de database verwijderd, maar op "niet actief" gezet (*soft delete*). Een niet-actieve auto:

- verschijnt niet meer in de zoekresultaten;
- kan niet meer worden gereserveerd of gewijzigd.

De oude reserveringen van de auto blijven zo bewaard, zodat een huurder zijn eerdere reserveringen kan blijven zien. Deze werkwijze heb ik in het eerste studiejaar geleerd bij het werken met databases.

Sleuteloverdracht

De casus beschrijft niet hoe de sleutel wordt overgedragen. Ik heb gekozen voor de werkwijze van platforms zoals SnappCar:

1. de huurder vraagt een reservering aan;
2. de eigenaar beoordeelt de aanvraag en keurt die goed of af;
3. na goedkeuring ziet de huurder de contactgegevens van de eigenaar;
4. bij het ophalen ontmoeten ze elkaar; de eigenaar controleert het rijbewijs en geeft de sleutel.

Zo bepaalt de eigenaar zelf aan wie hij zijn auto meegeeft. Een digitale controle van identiteit of rijbewijs valt buiten scope: daarvoor zijn externe diensten nodig en zou de applicatie gevoelige persoonsgegevens moeten opslaan (AVG).

1.5 User stories

Nr	User story	Eis
US-01	Als bezoeker wil ik een account aanmaken, zodat ik auto's kan aanbieden en huren.	FE-01
US-02	Als gebruiker wil ik inloggen, zodat ik bij mijn eigen auto's en reserveringen kan.	FE-02
US-03	Als eigenaar wil ik mijn auto aanbieden, zodat anderen hem kunnen huren als ik hem niet gebruik.	FE-03, FE-04
US-04	Als eigenaar wil ik de gegevens van mijn auto wijzigen, zodat bijvoorbeeld mijn tarieven kloppen.	FE-05
US-05	Als eigenaar wil ik mijn auto verwijderen, zodat hij niet meer verhuurd wordt.	FE-06
US-06	Als eigenaar wil ik foto's bij mijn auto zetten, zodat huurders zien wat ze huren.	FE-07
US-07	Als eigenaar wil ik de kosten per km en de TCO van mijn auto zien, zodat ik een eerlijk tarief kan bepalen.	FE-08, FE-09
US-08	Als bezoeker wil ik auto's zoeken en filteren, zodat ik snel een geschikte auto vind.	FE-10
US-09	Als bezoeker wil ik de details en foto's van een auto zien, zodat ik weet of de auto bij mij past.	FE-11
US-10	Als huurder wil ik een auto aanvragen voor een periode en de geschatte prijs zien, zodat ik weet wat het kost.	FE-12, FE-13
US-11	Als eigenaar wil ik aanvragen goedkeuren of afwijzen, zodat ik zelf bepaal aan wie ik mijn auto verhuur.	FE-14, FE-15, FE-18
US-12	Als huurder wil ik mijn reserveringen zien, zodat ik weet wanneer ik een auto heb.	FE-17
US-13	Als huurder wil ik een reservering annuleren, zodat de auto weer vrij is als ik hem niet nodig heb.	FE-16
US-14	Als huurder wil ik na goedkeuring de contactgegevens van de eigenaar zien, zodat we de sleuteloverdracht kunnen afspreken.	FE-19
US-15	Als gebruiker wil ik hulp van een AI-agent *(invulling volgt)*.	FE-20

1.6 Permissiematrix

Legenda: ✅ = mag, ❌ = mag niet. *Eigenaar* en *huurder* zijn ingelogde gebruikers in relatie tot een bepaalde auto of reservering.

Auto's

Actie	Bezoeker	Gebruiker	Eigenaar van de auto
Zoeken en bekijken	✅	✅	✅
Toevoegen	❌	✅	–
Wijzigen	❌	❌	✅
Verwijderen (zonder toekomstige reserveringen)	❌	❌	✅
Foto's toevoegen of verwijderen (periode 2)	❌	❌	✅
Kosten per km en TCO bekijken	❌	❌	✅

Reserveringen

Actie	Bezoeker	Gebruiker	Huurder van de reservering	Eigenaar van de auto
Aanvragen (niet voor eigen auto)	❌	✅	–	❌
Bekijken	❌	❌	✅	✅
Goedkeuren of afwijzen	❌	❌	❌	✅
Annuleren (vóór de start)	❌	❌	✅	❌
Contactgegevens eigenaar zien (na goedkeuring)	❌	❌	✅	–

Account

Actie	Bezoeker	Gebruiker
Account aanmaken	✅	–
Inloggen	✅	–
Eigen gegevens bekijken	❌	✅

1.7 Use cases

Overzicht

Nr	Use case	Actor	User story
UC-01	Account aanmaken	Bezoeker	US-01
UC-02	Inloggen	Bezoeker	US-02
UC-03	Auto aanbieden	Eigenaar	US-03
UC-04	Auto wijzigen	Eigenaar	US-04
UC-05	Auto verwijderen	Eigenaar	US-05
UC-06	Foto toevoegen *(periode 2, nog niet gerealiseerd)*	Eigenaar	US-06
UC-07	Kosten per km en TCO bekijken	Eigenaar	US-07
UC-08	Auto's zoeken	Bezoeker	US-08
UC-09	Auto bekijken	Bezoeker	US-09
UC-10	Reservering aanvragen	Huurder	US-10
UC-11	Aanvraag goedkeuren of afwijzen	Eigenaar	US-11
UC-12	Eigen reserveringen bekijken	Huurder	US-12, US-14
UC-13	Reservering annuleren	Huurder	US-13
UC-14	Advies van de AI-agent vragen *(nog niet gerealiseerd)*	Gebruiker	US-15

Hieronder zijn de belangrijkste use cases volledig uitgeschreven.

UC-02 Inloggen

Actor	Bezoeker
Voorwaarde	De bezoeker heeft een account.
Hoofdscenario	1. De bezoeker vult zijn e-mailadres en wachtwoord in. 2. Het systeem controleert of de combinatie klopt. 3. Het systeem geeft een inlogtoken (JWT) terug.
Alternatief	2a. De combinatie klopt niet: het systeem geeft een foutmelding (401) en geen token.
Resultaat	De gebruiker is ingelogd en kan met het token beveiligde acties uitvoeren.

UC-03 Auto aanbieden

Actor	Eigenaar (ingelogde gebruiker)
Voorwaarde	De gebruiker is ingelogd.
Hoofdscenario	1. De gebruiker kiest de soort auto (ICE, BEV of FCEV). 2. De gebruiker vult de gegevens in: merk, model, bouwjaar, kenteken, zitplaatsen, adres, tarieven, aanschafprijs en het verbruik dat bij de soort hoort. 3. Het systeem controleert de gegevens. 4. Het systeem slaat de auto op met de gebruiker als eigenaar. 5. Het systeem geeft de opgeslagen auto terug.
Alternatief	3a. Gegevens ontbreken of zijn ongeldig (bijvoorbeeld een negatief tarief): het systeem geeft een foutmelding (400) en slaat niets op. 3b. Het kenteken bestaat al: het systeem geeft een foutmelding (409).
Resultaat	De auto staat op het platform en is te vinden voor huurders.

UC-08 Auto's zoeken

Actor	Bezoeker (of gebruiker)
Voorwaarde	Geen.
Hoofdscenario	1. De bezoeker vult optioneel filters in: soort auto, plaats en maximale uurprijs. 2. Het systeem zoekt de actieve auto's die aan alle ingevulde filters voldoen. 3. Het systeem geeft de lijst met auto's terug.
Alternatief	2a. Er zijn geen auto's die voldoen: het systeem geeft een lege lijst terug. 2b. Een filter is ongeldig (bijvoorbeeld een onbekende soort auto of een uurprijs die geen getal is): het systeem geeft een foutmelding (400).
Resultaat	De bezoeker ziet de auto's die aan zijn wensen voldoen.
Periode 2	Filteren op een periode (alleen auto's die dan nog vrij zijn) is nog niet gerealiseerd en volgt in periode 2.

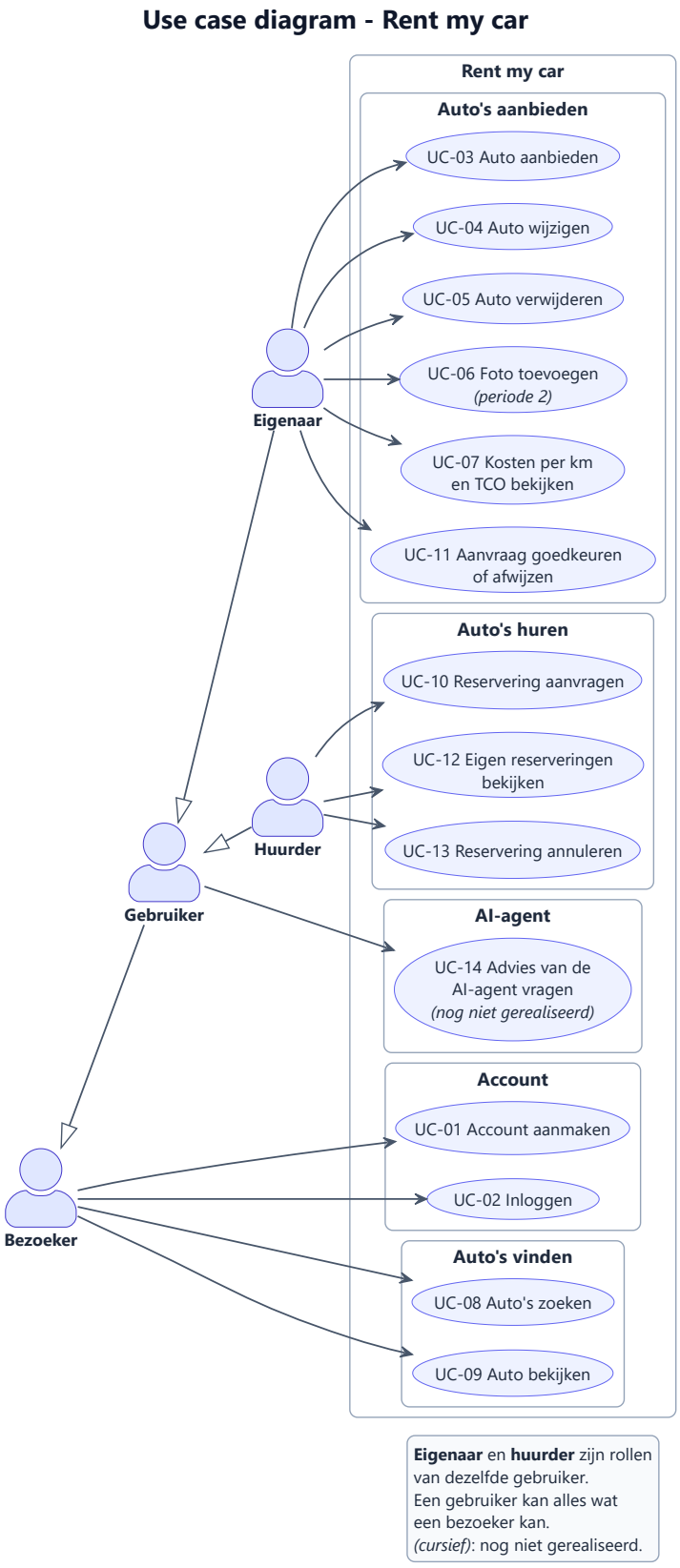
UC-10 Reservering aanvragen

Actor	Huurder (ingelogde gebruiker)
Voorwaarde	De gebruiker is ingelogd en heeft een auto gevonden.
Hoofdscenario	1. De huurder kiest een startmoment, een eindmoment en het verwachte aantal kilometers. 2. Het systeem controleert de regels voor een reservering. 3. Het systeem berekent de geschatte huurprijs. 4. Het systeem slaat de reservering op met status `AANGEVRAAGD`. 5. Het systeem geeft de reservering met de geschatte prijs terug.
Alternatief	2a. Het eindmoment ligt niet na het startmoment, de duur is korter dan 1 uur of de start ligt in het verleden: foutmelding (400). 2b. De huurder probeert zijn eigen auto te reserveren: foutmelding (400). 2c. De auto bestaat niet of is niet actief: foutmelding (404).
Resultaat	Er ligt een aanvraag die de eigenaar kan beoordelen.

UC-11 Aanvraag goedkeuren of afwijzen

Actor	Eigenaar
Voorwaarde	De eigenaar is ingelogd en er is een aanvraag voor een van zijn auto's.
Hoofdscenario (goedkeuren)	1. De eigenaar bekijkt de aanvragen voor zijn auto. 2. De eigenaar kiest een aanvraag en keurt die goed. 3. Het systeem controleert dat de status `AANGEVRAAGD` is en de start nog niet is bereikt. 4. Het systeem controleert dat er geen goedgekeurde reservering voor deze auto is die overlapt. 5. Het systeem zet de status op `GOEDGEKEURD`.
Alternatief	2a. De eigenaar wijst de aanvraag af: het systeem zet de status op `AFGEWEZEN`. 3a. De aanvraag is niet meer `AANGEVRAAGD` (bijvoorbeeld geannuleerd): foutmelding (409). 4a. Er is al een goedgekeurde reservering die overlapt: het systeem weigert het goedkeuren (409). De aanvraag blijft `AANGEVRAAGD`, zodat de eigenaar die kan afwijzen. 2b. De gebruiker is niet de eigenaar van de auto: foutmelding (403).
Resultaat	De huurder weet of hij de auto kan gebruiken. Bij goedkeuring ziet de huurder de contactgegevens van de eigenaar.

1.8 Use case diagram



Use case diagram

Bron: `puml/use-case-diagram.puml`

2. Ontwerp

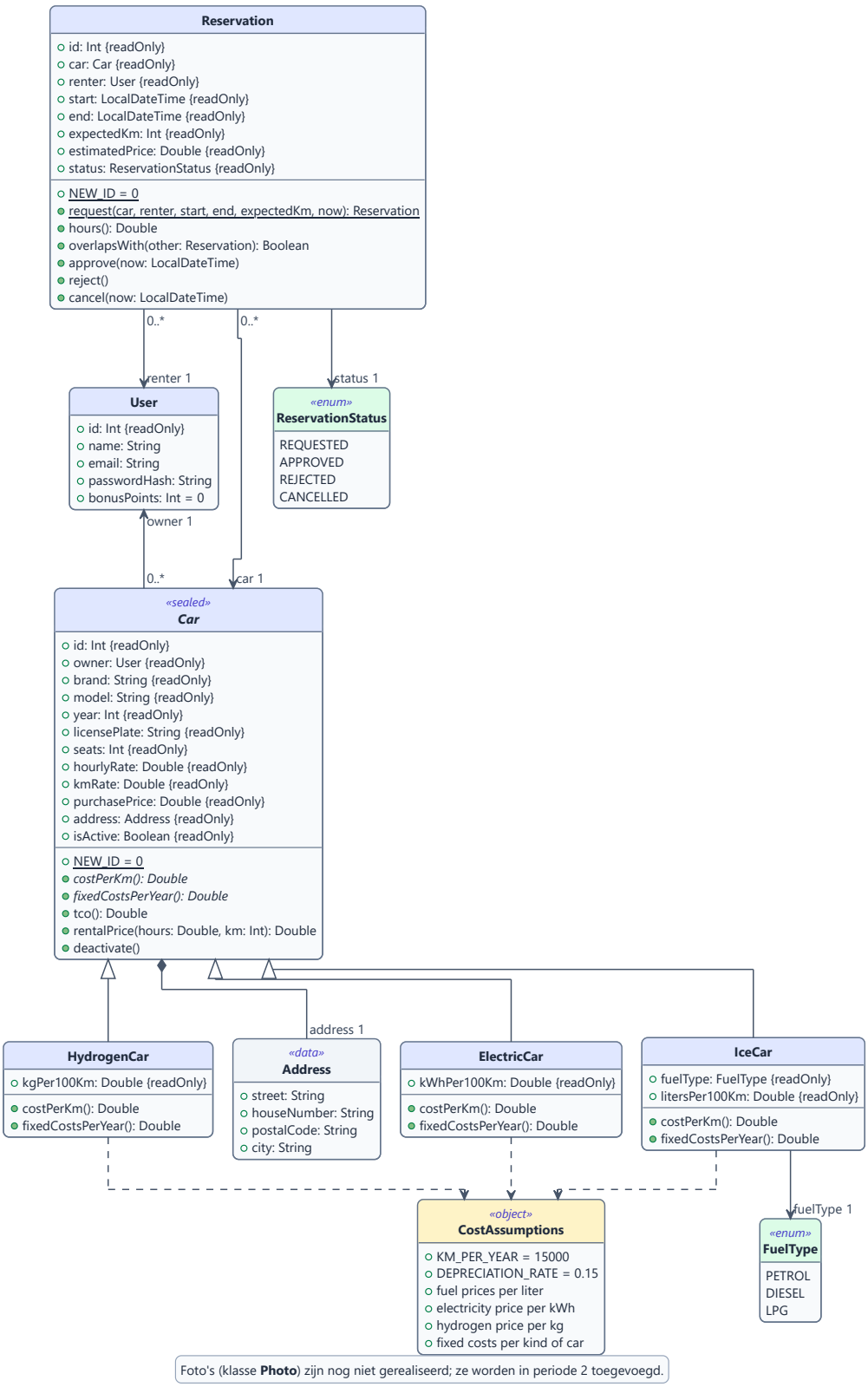
In dit hoofdstuk staat hoe de applicatie is opgebouwd. Eerst het domeinmodel: de klassen die de wereld van Rent my car beschrijven. Daarna de indeling in packages en het databasemodel.

Twee begrippen die in dit hoofdstuk vaak terugkomen:

- **Domein:** de wereld waar de applicatie over gaat: auto's, gebruikers en reserveringen, en de regels die daarbij horen. Bijvoorbeeld: "een reservering duurt minimaal 1 uur". Die regels gelden ook zonder computer. Het *domeinmodel* zijn de klassen die deze wereld beschrijven.
- **Repository:** een technisch onderdeel dat domeinobjecten opslaat in de database en weer ophaalt. Een repository bedenkt geen regels, maar bewaart alleen. De repositories staan in de architectuur in hoofdstuk 3.1.

2.1 Domeinmodel (klassendiagram)

Klassendiagram - Domein



Domein-klassendiagram

Bron: `puml/domain-class-diagram.puml`

De namen in de code zijn Engels. De belangrijkste vertalingen:

Code	Betekenis
`Car`, `IceCar`, `ElectricCar`, `HydrogenCar`	Auto, ICE-auto, BEV (elektrisch), FCEV (waterstof)
`User`	Gebruiker (als eigenaar of als huurder)
`Reservation`	Reservering
`REQUESTED`, `APPROVED`, `REJECTED`, `CANCELLED`	Aangevraagd, goedgekeurd, afgewezen, geannuleerd
`PETROL`, `DIESEL`, `LPG`	Benzine, diesel, LPG

Toelichting per klasse

`User` is een gebruiker van het platform. Een gebruiker kan eigenaar zijn van auto's en huurder van reserveringen. Het wachtwoord wordt alleen gehasht opgeslagen (`passwordHash`). De bonuspunten beginnen op 0 en worden in periode 2 gebruikt.

`Car` bevat alles wat voor elke auto geldt: merk, model, kenteken, tarieven, adres en de eigenaar. `Car` is een *sealed class*. Je kunt dus geen "losse" `Car` maken, maar alleen een van de drie soorten. Bij het maken controleert de klasse de invoer met `require()` : merk, model en kenteken zijn verplicht, er is minstens 1 zitplaats en tarieven en aanschafprijs zijn niet negatief. De klasse heeft twee abstracte functies die elke soort zelf invult:

- `costPerKm()` : de kosten per kilometer, berekend met het verbruik en de energieprijs van die soort;
- `fixedCostsPerYear()` : de vaste kosten per jaar (verzekering, wegenbelasting en onderhoud) van die soort.

Daarnaast heeft `Car` :

- `rentalPrice(hours, km)` : de huurprijs voor een aantal uren en kilometers (uurtarief × uren + km-tarief × km);
- `isActive` en `deactivate()` : een auto wordt niet echt verwijderd, maar op niet actief gezet (zie hoofdstuk 1.4). `isActive` kan alleen via `deactivate()` veranderen (`private set`);
- `NEW_ID` in het *companion object*: het id (0) van een auto die nog niet is opgeslagen. De database geeft bij het opslaan het echte id.

De functie `tco()` staat één keer in `Car` en gebruikt de twee abstracte functies. Daardoor werkt de TCO-berekening voor alle soorten auto's, zonder dat de code drie keer hoeft te worden geschreven. De formules staan in `aannames.md` .

`IceCar` , `ElectricCar` en `HydrogenCar` zijn de drie soorten auto's uit de casus (ICE, BEV en FCEV). Ze voegen hun eigen verbruik toe en berekenen daarmee hun kosten per kilometer:

Klasse	Eigen gegevens	Kosten per km
`IceCar`	`fuelType` , `litersPer100Km`	liters per 100 km × literprijs van de brandstofsoort ÷ 100
`ElectricCar`	`kWhPer100Km`	kWh per 100 km × stroomprijs ÷ 100
`HydrogenCar`	`kgPer100Km`	kg per 100 km × waterstofprijs ÷ 100

`FuelType` is een *enum* met de drie brandstofsoorten van een ICE-auto. De enum staat in de gedeelde module `core`, omdat de app in periode 2 ook moet weten welke brandstofsoorten er zijn. De prijzen staan niet in de enum maar in `CostAssumptions`, omdat de app die niet nodig heeft.

`Address` is het adres waar de auto staat. Het is een *data class*: het bevat alleen gegevens.

`Photo` *(periode 2, nog niet gerealiseerd)*: een foto van een auto. Het plan is dat alleen de bestandsnaam in de database komt en het bestand zelf op de schijf van de server. Deze klasse staat daarom nog niet in het klassendiagram.

`Reservation` is een reservering van een auto door een huurder, met een start- en eindmoment, het verwachte aantal kilometers en een status. De klasse bevat de regels die alleen over de reservering zelf gaan:

- een nieuwe reservering wordt gemaakt met de functie `Reservation.request(...)` in het *companion object*. Die controleert eerst alle regels (de huurder is niet de eigenaar, de start ligt in de toekomst, het eindmoment ligt na het startmoment, de duur is minimaal 1 uur, het verwachte aantal kilometers is niet negatief), berekent de prijs en geeft een reservering met status `REQUESTED` terug. Zo kan er nooit een reservering bestaan die niet aan de regels voldoet;
- `estimatedPrice` is de geschatte huurprijs. Die wordt bij het aanvragen één keer berekend met `car.rentalPrice()` en daarna vastgelegd. Als de eigenaar later zijn tarieven aanpast, blijft de prijs van de reservering gelijk;
- `hours()` geeft het aantal uren van de reservering;
- `overlapsWith()` controleert of twee reserveringen in tijd overlappen;
- `approve()`, `reject()` en `cancel()` veranderen de status, maar alleen als dat volgens de levenscyclus mag (zie hoofdstuk 1.4). De status zelf heeft een `private set`.

De functies die van de tijd afhangen (`request()`, `approve()` en `cancel()`) krijgen de huidige tijd mee als parameter (`now`), in plaats van zelf de klok te lezen. Daardoor kunnen ze met een vaste tijd worden getest: bijvoorbeeld "wat gebeurt er als de start al voorbij is?".

`ReservationStatus` is een *enum* met de vier statussen van een reservering.

`CostAssumptions` is een *object* (er bestaat er maar één van) met alle vaste waarden uit `aannames.md`: het aantal kilometers per jaar, het afschrijvingspercentage, de prijzen van brandstof, stroom en waterstof, en de vaste kosten per soort auto.

2.2 Toelichting op de OO-keuzes

Gebruikte OO-concepten

Concept	Waar in het domeinmodel	Waarom
Overerving	`IceCar`, `ElectricCar` en `HydrogenCar` erven van `Car`.	De drie soorten hebben veel gegevens gemeenschappelijk. Die staan één keer in `Car`.
Abstracte klasse (sealed class)	`Car`	Een auto is altijd een van de drie soorten. Een *sealed class* legt vast dat er precies deze drie soorten zijn. Dit is dezelfde opbouw als `Property` in de HousingMarket-workshop van les 3.

Concept	Waar in het domeinmodel	Waarom
Polymorfisme	`costPerKm()` en `fixedCostsPerYear()`	Overal in de code wordt dezelfde functie aangeroepen; elke soort auto voert zijn eigen versie uit. `tco()` hoeft niet te weten welke soort auto het is.
Compositie	`Car` ◆ `Address` (en in periode 2 `Car` ◆ `Photo`)	Een adres hoort bij precies één auto en bestaat niet zonder die auto. Het wordt nooit los van de auto gebruikt.
Associatie	`Car` → `User` (eigenaar), `Reservation` → `Car` en `Reservation` → `User` (huurder)	Deze objecten bestaan los van elkaar. Een gebruiker blijft bestaan als zijn auto wordt verwijderd.
Enum	`FuelType`, `ReservationStatus` (en `CarType` in de DTO's)	Een vaste, kleine lijst mogelijkheden.
Object (singleton)	`CostAssumptions`	Alle vaste getallen staan op één plek. Als een prijs verandert, hoeft dat maar op één plek te worden aangepast.
Encapsulatie	De status van `Reservation`	De status kan alleen veranderen via `approve()`, `reject()` en `cancel()`. Die functies controleren eerst of de overgang mag. Zo kan een afgewezen reservering bijvoorbeeld nooit meer worden goedgekeurd.
Companion object	`Reservation.request(...)`, `Car.NEW_ID`, `Reservation.NEW_ID`	Iets dat bij de klasse hoort en niet bij één object. `request()` maakt een nieuwe reservering en controleert eerst alle regels; dit is een eenvoudige vorm van het *factory*-patroon.
Data class	`Address`	Een klasse die alleen gegevens bevat. Kotlin maakt automatisch onder andere `equals()` en `toString()`.

Keuze: de overlapcontrole zit in `Reservation`

Bij het goedkeuren van een aanvraag moet worden gecontroleerd of de auto in die periode al is verhuurd. Dit bestaat uit twee stappen:

1. **de regel:** overlappen twee periodes? Dit is pure logica en zit in `Reservation.overlapsWith()`;
2. **het opzoeken** van de andere goedgekeurde reserveringen van dezelfde auto. Daarvoor is de database nodig, dus dit gebeurt in de service (hoofdstuk 3).

Twee periodes overlappen als de ene begint voordat de andere eindigt, en andersom. Een reservering die eindigt op het moment dat de volgende begint, overlapt dus niet.

Deze keuze is vergelijkbaar met de HousingMarket-workshop uit les 3: daar zit de regel "wordt dit bod geaccepteerd?" (`isAccepted()`) ook in de klasse `Property` zelf. Het voordeel is dat de regel eenvoudig met unit tests te controleren is, zonder database.

Keuze: de interfaces staan bij de repositories

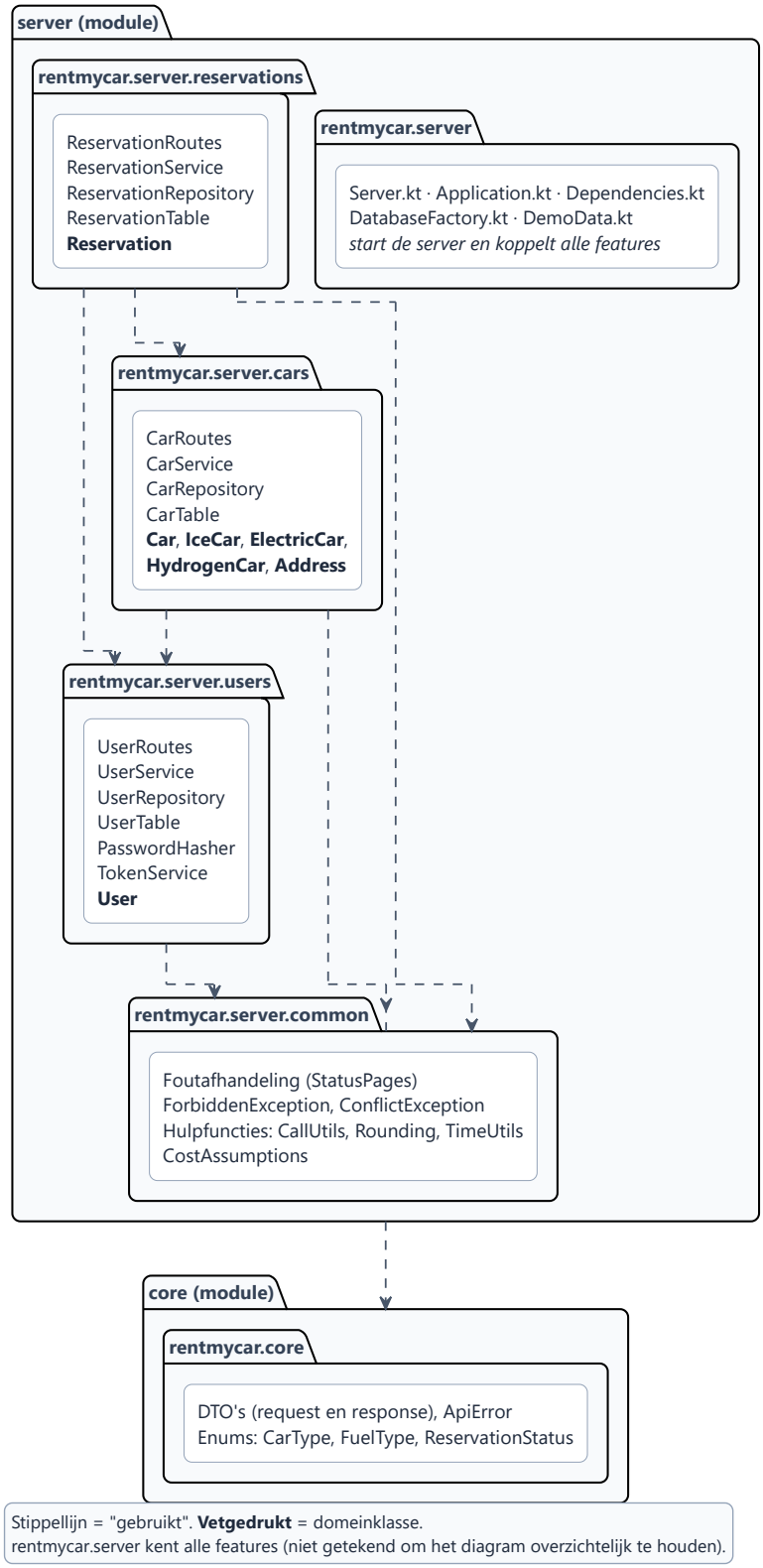
In het domeinmodel staat geen interface. Er is in het domein geen onderdeel waar een interface echt iets toevoegt, en een verzonnen interface maakt het ontwerp alleen ingewikkelder. Interfaces worden wel gebruikt bij de repositories (bijvoorbeeld `CarRepository`). Daar zorgen ze ervoor dat de rest van de code niet afhangt van de database, zodat in de tests een nep-versie kan worden gebruikt. Dit staat in het architectuurdiagram in hoofdstuk 3.1. Deze werkwijze komt uit les 3 (`EmployeeRepository`), les 7 (`FakeTaskRepository`) en de voorbeeldapplicatie van de docent (`FakeUserRepository`).

Keuze: bedragen als `Double`

Bedragen worden opgeslagen als `Double`. Dat is eenvoudig, maar kan kleine afrondingsverschillen geven. Bij het versturen worden bedragen daarom afgerond met de functie `rounded()` uit `common/Rounding.kt`: de huurprijs en de TCO op twee decimalen, en de kosten per km op vier decimalen (bijvoorbeeld € 0,1452), omdat dat bedrag zo klein is. Voor een echte betaalapplicatie zou een type zonder afrondingsfouten nodig zijn; omdat er in deze applicatie geen echte betalingen zijn, is `Double` voldoende.

2.3 Packagediagram

Packagediagram - Server en core



Packagediagram

Bron: `puml/package-diagram.puml`

De code van de server is ingedeeld **per feature**: elk onderdeel van de casus heeft een eigen package. Binnen elke feature staan dezelfde lagen: routes, service en repository (zie hoofdstuk 3.1).

Package	Inhoud
`rentmycar.server`	Het starten van de server en het inlezen van de instellingen (`Server.kt`), het installeren van de plugins en het koppelen van de routes (`Application.kt`), het aanmaken van alle services en repositories (`Dependencies.kt`), het verbinden met de database en het maken van de tabellen (`DatabaseFactory.kt`) en het vullen van een lege database met testdata (`DemoData.kt`).
`rentmycar.server.users`	Gebruikers: registreren, inloggen, het domeinobject `User` , de tabel `UserTable` , het hashen van wachtwoorden (`PasswordHasher`) en het maken en controleren van tokens (`TokenService` , met de instellingen in `JwtConfig`).
`rentmycar.server.cars`	Auto's: aanbieden, zoeken, wijzigen, verwijderen, kosten berekenen, de tabel `CarTable` en de domeinklassen `Car` , `IceCar` , `ElectricCar` , `HydrogenCar` en `Address` . Foto's (`Photo`) volgen in periode 2.
`rentmycar.server.reservations`	Reserveringen: aanvragen, goedkeuren, afwijzen, annuleren, de tabel `ReservationTable` en het domeinobject `Reservation` .
`rentmycar.server.common`	Onderdelen die alle features gebruiken: de foutafhandeling (`StatusPages.kt` en `Exceptions.kt`), hulpfuncties voor een verzoek (`CallUtils.kt` : het gebruikers-id uit het token en een getal uit de URL), afronden (`Rounding.kt`), tijd (`TimeUtils.kt`) en `CostAssumptions` .
`rentmycar.core`	De gedeelde module met de DTO's, `ApiError` en de enums `CarType` , `FuelType` en `ReservationStatus` .

Elke feature heeft ook een bestand `...Mapping.kt` met de functie `toResponse()` , die een domeinobject omzet naar een DTO. Het omzetten van een databaserij naar een domeinobject gebeurt in de repository zelf.

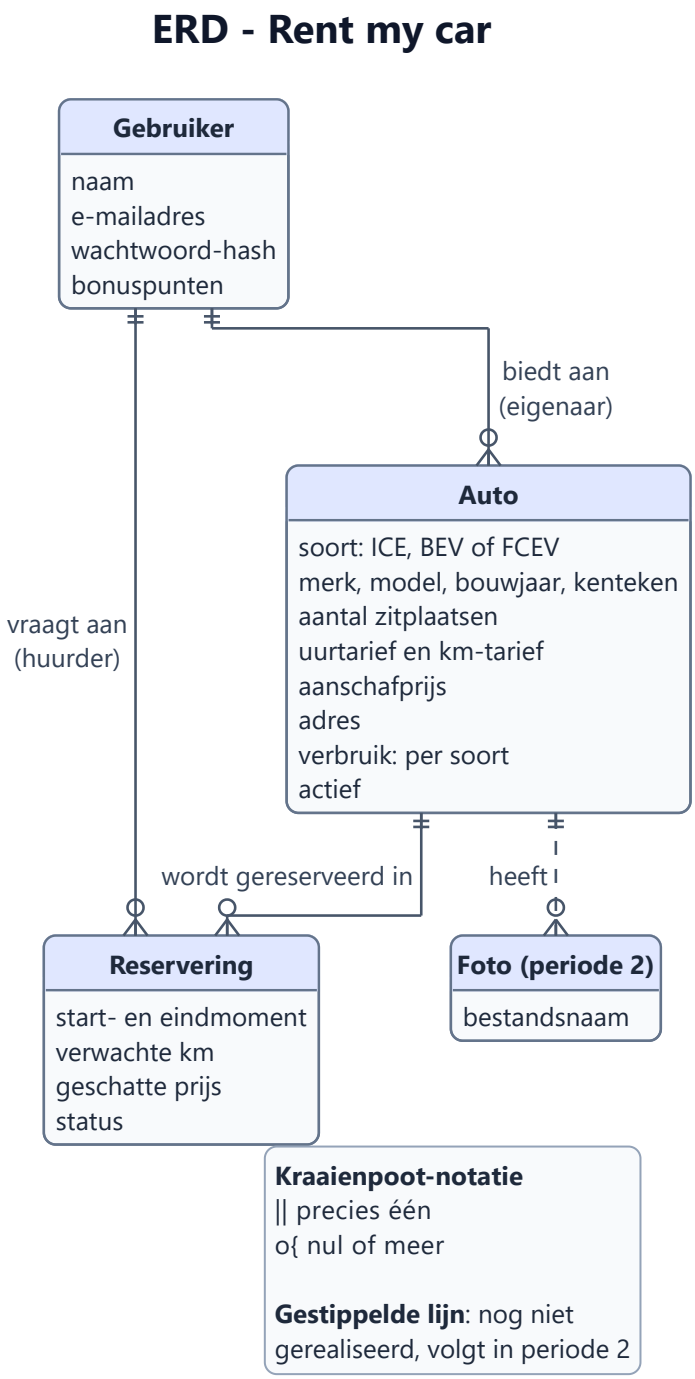
Afhankelijkheden:

- `reservations` gebruikt `cars` en `users` , omdat een reservering een auto en een huurder heeft;
- `cars` gebruikt `users` , omdat een auto een eigenaar heeft;
- `users` gebruikt geen andere feature. Er zijn dus geen kringen: de afhankelijkheden lopen maar één kant op;
- alle features gebruiken `common` en de gedeelde module `core` .

Wat staat in `core` en wat in `server` ?

- In `core` staan alleen de **DTO's** (*Data Transfer Objects*: de gegevens die als JSON tussen de app en de server worden verstuurd) en de **enums**. De Android-app gebruikt in periode 2 precies dezelfde klassen. Zo kunnen de app en de server niet van elkaar verschillen.
- De **domeinklassen** met hun regels en berekeningen staan in `server` . De regels worden alleen op de server uitgevoerd; de app krijgt de uitkomsten via de DTO's.

2.4 Entiteit-relatiediagram (ERD)



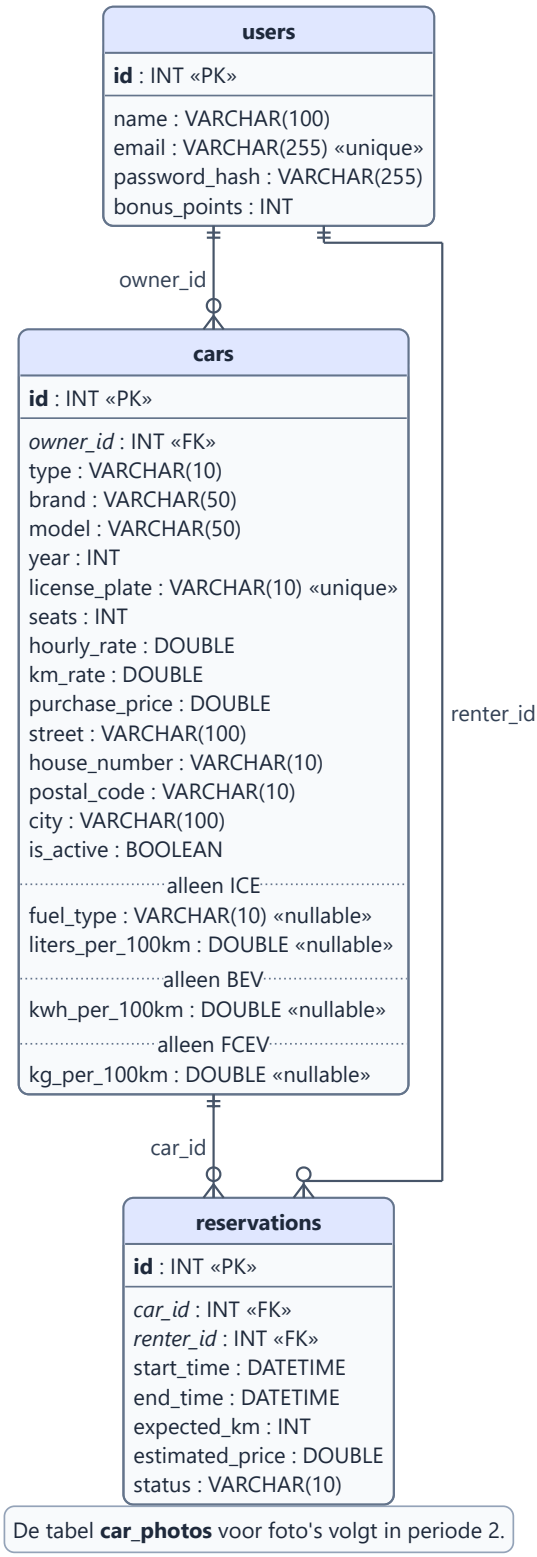
ERD

Bron: `puml/erd.puml`

Het ERD laat zien welke gegevens de applicatie bewaart en hoe ze samenhangen, nog zonder technische details. Het diagram gebruikt de kraaienpoot-notatie.

Relatie	Betekenis
Gebruiker – Auto	Een gebruiker biedt nul of meer auto's aan. Een auto heeft precies één eigenaar.
Gebruiker – Reservering	Een gebruiker vraagt nul of meer reserveringen aan. Een reservering heeft precies één huurder.
Auto – Reservering	Een auto heeft nul of meer reserveringen. Een reservering is voor precies één auto.
Auto – Foto	Een auto heeft nul of meer foto's. Een foto hoort bij precies één auto. *Volgt in periode 2 (gestippeld in het ERD).*

2.5 Relationeel databasemodel



Databasemodel

Bron: `puml/database-model.puml`

De gegevens worden opgeslagen in drie tabellen: `users`, `cars` en `reservations`. In de code worden deze tabellen beschreven met de library Exposed (`UserTable`, `CarTable` en `ReservationTable`). `DatabaseFactory` maakt de tabellen bij het starten als ze nog niet bestaan. De database staat in het bestand `data/rentmycar.mv.db`, zodat de gegevens na een herstart bewaard blijven; de tests gebruiken een database in het geheugen.

In periode 2 komt er een vierde tabel bij: `car_photos`, voor de foto's van een auto.

Vertaling van het OO-model naar tabellen

Een database kent geen klassen, overerving of lijsten, maar alleen tabellen met kolommen. Het domeinmodel uit 2.1 is daarom als volgt vertaald:

In Kotlin	In de database	Voorbeeld	Bron
Klasse	Tabel	`User` → `users`	Algemeen
Eigenschap	Kolom	`licensePlate` → `license_plate`	Algemeen
Associatie	Vreemde sleutel (FK)	De eigenaar van `Car` → `owner_id` verwijst naar `users`	Algemeen
Compositie met één object	Kolommen in dezelfde tabel	`Address` → `street`, `house_number`, `postal_code`, `city` in `cars`	Eigen keuze
Compositie met een lijst (periode 2)	Aparte tabel met een FK	`photos` → tabel `car_photos` met `car_id`	Zoals `IncidentImagesTable` in de demo van de docent
Overerving	Eén tabel met een kolom `type`	`IceCar`, `ElectricCar`, `HydrogenCar` → `cars`	Eigen keuze, zie hieronder
Enum	De naam als tekst	`FuelType.PETROL` → `'PETROL'`	Les 7: `enumerationByName`
Tijdstip	`DATETIME`	`start` → `start_time`	Algemeen

De kolommen `start` en `end` heten in de database `start_time` en `end_time`, omdat `end` in SQL een gereserveerd woord is.

Keuze: overerving in één tabel

Voor het opslaan van de drie soorten auto's zijn drie bekende manieren (uit *Patterns of Enterprise Application Architecture* van Martin Fowler). Deze manieren komen niet uit het lesmateriaal; de keuze is mijn eigen afweging.

Manier	Hoe	Voordelen	Nadelen
A. Eén tabel (*Single Table Inheritance*) ✅	Alle auto's in de tabel `cars`, met een kolom `type` en de kolommen van alle soorten	Eenvoudig: één tabel, zoeken over alle auto's is één query, een reservering verwijst naar één tabel	Sommige kolommen zijn leeg, afhankelijk van de soort
B. Een tabel per soort plus een gedeelde tabel (*Class Table Inheritance*)	Gedeelde gegevens in `cars`, eigen gegevens in `ice_cars`, `electric_cars` en `hydrogen_cars`	Geen lege kolommen	Vier tabellen; elke query moet tabellen samenvoegen (joins)
C. Een losse tabel per soort (*Concrete Table Inheritance*)	Drie tabellen met elk alle kolommen	Geen lege kolommen	Zoeken over alle auto's is lastig, en een reservering zou naar drie tabellen moeten kunnen verwijzen

Ik heb gekozen voor **A**, omdat dit het eenvoudigst te bouwen en te begrijpen is. De lege kolommen zijn een bewuste afweging: er zijn maar vier extra kolommen. Bij het uitlezen bepaalt de kolom `type` welke klasse wordt gemaakt:

```
// Make the right kind of car from a database row (shortened)
val type = row[CarTable.type]
if (type == CarType.ICE) {
    return IceCar(...)
}
if (type == CarType.BEV) {
    return ElectricCar(...)
}
return HydrogenCar(...)
```

De kolom `type` is de enum `CarType`, opgeslagen als tekst (`enumerationByName`). Dit staat in de functie `toCar()` van `ExposedCarRepository`.

Keuze: auto's worden niet echt verwijderd

De tabel `cars` heeft een kolom `is_active`. Als een eigenaar zijn auto verwijdert, wordt `is_active` op `false` gezet en blijft de rij bestaan (*soft delete*). Daardoor blijven de reserveringen die naar deze auto verwijzen geldig, en kan een huurder zijn oude reserveringen blijven zien. Bij het zoeken worden alleen auto's met `is_active = true` getoond. Een niet-actieve auto geeft bij opvragen, wijzigen, verwijderen en reserveren een 404.

Keuze: de huurprijs wordt vastgelegd

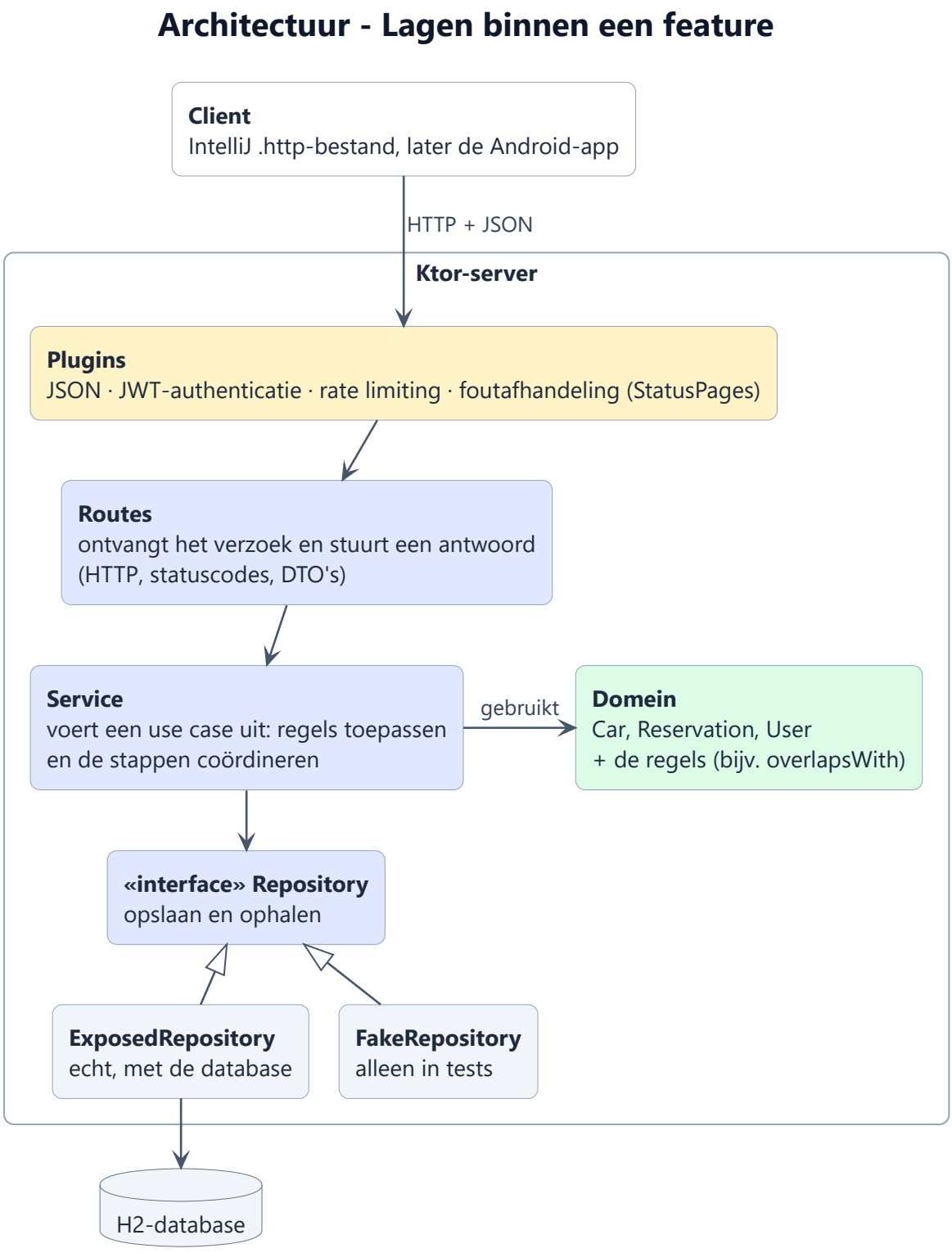
De tabel `reservations` heeft een kolom `estimated_price`. De prijs wordt bij het aanvragen berekend en opgeslagen. Als de eigenaar later zijn tarieven verandert, blijft de prijs van een bestaande reservering hetzelfde, net als bij een echte boeking.

Unieke waarden

- `users.email` is uniek: met één e-mailadres kun je maar één account maken.
- `cars.license_plate` is uniek: een auto kan maar één keer worden aangeboden.

3. Web API

3.1 Architectuur



Architectuur

Bron: `puml/architecture.puml`

Lagen

Elk verzoek dat bij de server binnenkomt, gaat door dezelfde lagen. Elke laag heeft één verantwoordelijkheid:

Laag	Verantwoordelijkheid	Voorbeeld
Plugins	Algemene zaken voor alle verzoeken: JSON omzetten, controleren of de gebruiker is ingelogd (JWT) en fouten omzetten naar een nette foutmelding.	Een verzoek zonder geldig token krijgt status 401.
Routes	Alleen HTTP: het verzoek uitlezen, de service aanroepen en een antwoord met de juiste statuscode terugsturen. De route zet het antwoord om naar een DTO met `toResponse()`.	`PATCH /api/reservations/{id}/status` roept `ReservationService.changeStatus()` aan.
Service	Een use case uitvoeren: de stappen coördineren en de regels toepassen.	Bij goedkeuren: de aanvraag ophalen, controleren dat de gebruiker de eigenaar is, controleren op overlap en de nieuwe status opslaan.
Domein	De klassen en regels van Rent my car (hoofdstuk 2). Het domein weet niets van HTTP of de database.	`reservation.overlapsWith(other)`
Repository	Domeinobjecten opslaan en ophalen. De service praat alleen met de *interface*; de echte versie gebruikt Exposed en de H2-database.	`ExposedReservationRepository`

Doordat de service alleen de interface van de repository kent, kan in de tests een nep-versie (`FakeRepository`) worden meegegeven. Dan kunnen de routes en services worden getest zonder database (zie hoofdstuk 4).

Indeling per feature

De lagen zitten niet in één grote map per laag, maar in een package per feature (`users` , `cars` , `reservations` ; zie het packagediagram in hoofdstuk 2.3). Alles wat bij auto's hoort, staat dus bij elkaar. Binnen elke feature zijn de lagen wel te herkennen aan de bestandsnamen:

```
server/src/
├── Server.kt           main(): laadt de instellingen en start de server
├── Application.kt      installeert de plugins en koppelt de routes
├── Dependencies.kt     maakt alle repositories en services aan
├── DatabaseFactory.kt  verbindt met H2 en maakt de tabellen
├── DemoData.kt         vult een lege database met testdata
├── users/
├── cars/
│   ├── CarRoutes.kt       routes
│   ├── CarService.kt      service
│   ├── CarRepository.kt   interface + ExposedCarRepository
│   ├── CarTable.kt        de tabel voor Exposed
│   ├── CarMapping.kt      domeinobject -> DTO
│   └── Car.kt, IceCar.kt, ElectricCar.kt, HydrogenCar.kt, Address.kt   domein
├── reservations/
└── common/
```

Onderdelen aan elkaar koppelen (dependency injection)

In `Dependencies.kt` worden alle repositories en services één keer aangemaakt en als parameter aan elkaar doorgegeven:

```
// Makes all repositories and services once and passes them to each other (shortened)
val userRepository = ExposedUserRepository(database)
val carRepository = ExposedCarRepository(database, userRepository)
val reservationRepository = ExposedReservationRepository(database, carRepository,
userRepository)

val carService = CarService(carRepository, userRepository, reservationRepository)
val reservationService = ReservationService(reservationRepository, carRepository,
userRepository)
```

Hiervoor wordt geen apart framework gebruikt. Het is gewone Kotlin-code die in volgorde wordt uitgevoerd, dus makkelijk te volgen. In een test wordt op dezelfde manier een service gemaakt, maar dan met fake repositories (`FakeCarRepository` , `FakeReservationRepository` en `FakeUserRepository`).

Herkomst van de keuzes

Keuze	Bron	Afwijking?
Scheiding in routes, service, domein en repository	Leeruitkomst 1 ("scheidt verantwoordelijkheden: routing, use-cases/service, domein, persistence"); les 3 en 4 ("klassieke 3-lagen: route, services en repository"); les 6 ("toepassen service layer")	Nee
Indeling per feature (`users`, `cars`, `reservations`)	Gastles Ktor (`nomisRev/avans-toolchain-and-ktor-2026`, *architecture*: "Keep feature code together"); voorbeeldapplicatie van de docent (`auth`, `users`, `incidents`)	Nee
`Server.kt`, `Application.kt` en `Dependencies.kt`	Gastles Ktor ("Bootstrap constructs; application wires"); voorbeeldapplicatie van de docent (`Dependencies.kt`)	Nee
Interfaces alleen voor repositories	Gastles Ktor: "Don't create an interface for every class automatically"	Nee
Fake repositories in tests	Les 7 (`FakeTaskRepository`); voorbeeldapplicatie van de docent (`FakeUserRepository`)	Nee
Geen Koin voor dependency injection	Gastles Ktor: "dependency injection without a framework"	**Ja.** Op Brightspace (*Handige overzichten*) wordt Koin aanbevolen. Ik volg het advies van de gastles, omdat het zonder framework eenvoudiger is en voor deze applicatie voldoende.
Een service voor elke feature	Les 6 ("toepassen service layer")	**Klein verschil.** De gastles zegt een service pas toe te voegen als er een echte use case is. Alle drie de features hebben regels (eigenaarschap, overlap, wachtwoorden), dus een service is hier steeds nodig.
Domeinklassen in `server`, alleen DTO's en enums in `core`	Voorbeeldapplicatie van de docent: DTO's en enums in de gedeelde module	**Ja.** In de demo van de docent staan ook de domeinmodellen in de gedeelde module. Ik houd de domeinklassen op de server, omdat de regels en berekeningen alleen daar worden uitgevoerd; de app krijgt de uitkomsten via de DTO's.
Module heet `core` in plaats van `shared`	De wizard maakte de module `core` aan; `app/shared` bestaat al voor de app	Alleen een andere naam

3.2 Endpoints

Alle URL's beginnen met `/api`, net als in de voorbeeldapplicatie van de docent. De gegevens worden verstuurd als JSON. 🔒 betekent dat een geldig inlogtoken nodig is (zie 3.4).

Gebruikers

Methode en URL	Wat	Gelukt	Use case
`POST /api/users/register`	Account aanmaken	201 + gebruiker	UC-01
`POST /api/users/login`	Inloggen	200 + token	UC-02
`GET /api/users/me` 🔒	Eigen gegevens bekijken	200 + gebruiker	–

Auto's

Methode en URL	Wat	Gelukt	Use case
`GET /api/cars`	Actieve auto's zoeken, met optionele filters (zie hieronder)	200 + lijst	UC-08
`GET /api/cars/{id}`	Details van een actieve auto	200 + auto	UC-09
`GET /api/cars/mine` 🔒	Eigen auto's bekijken	200 + lijst	–
`POST /api/cars` 🔒	Auto aanbieden	201 + auto	UC-03
`PUT /api/cars/{id}` 🔒 eigenaar	Auto wijzigen	200 + auto	UC-04
`DELETE /api/cars/{id}` 🔒 eigenaar	Auto op niet actief zetten	204	UC-05
`GET /api/cars/{id}/costs` 🔒 eigenaar	Kosten per km en TCO per jaar	200 + kosten	UC-07

`GET /api/cars/mine` toont ook de eigen auto's die niet meer actief zijn.

Gepland voor periode 2 (nog niet gerealiseerd):

Methode en URL	Wat	Gelukt	Use case
`POST /api/cars/{id}/photos` 🔒 eigenaar	Foto uploaden (`multipart/form-data`)	201 + foto	UC-06
`DELETE /api/cars/{id}/photos/{photoId}` 🔒 eigenaar	Foto verwijderen	204	UC-06
`GET /api/photos/{fileName}`	Foto bekijken	200 + afbeelding	UC-09

Zoekfilters bij `GET /api/cars`; alle filters zijn optioneel en kunnen worden gecombineerd:

Parameter	Betekenis	Voorbeeld
`type`	Soort auto (hoofdletters maken niet uit)	`ICE` , `BEV` of `FCEV`
`city`	Plaats (hoofdletters maken niet uit)	`Breda`
`maxHourlyRate`	Maximale uurprijs	`10.0`

Voorbeeld: `GET /api/cars?type=BEV&city=Breda&maxHourlyRate=10`

Een onbekende soort auto of een uurprijs die geen getal is, geeft 400. Filteren op beschikbaarheid in een periode (`from` en `to`) is nog niet gerealiseerd en volgt in periode 2.

Reserveringen

Methode en URL	Wat	Gelukt	Use case
`POST /api/reservations` 🔒	Reservering aanvragen	201 + reservering met geschatte prijs	UC-10
`GET /api/reservations/mine` 🔒	Eigen reserveringen als huurder	200 + lijst	UC-12
`GET /api/cars/{id}/reservations` 🔒 eigenaar	Aanvragen en reserveringen voor een eigen auto	200 + lijst	UC-11
`GET /api/reservations/{id}` 🔒 huurder of eigenaar	Details; na goedkeuring met de contactgegevens van de eigenaar	200 + reservering	UC-12
`PATCH /api/reservations/{id}/status` 🔒	Status wijzigen: goedkeuren, afwijzen of annuleren	200 + reservering	UC-11, UC-13

Het wijzigen van de status gaat via één endpoint, zoals in de voorbeeldapplicatie van de docent (`PATCH /{incidentId}/status`). De service controleert wie wat mag:

Nieuwe status	Wie	Voorwaarde
`APPROVED`	Eigenaar van de auto	Status is `REQUESTED` , de start is nog niet bereikt en er is geen overlap met een andere goedgekeurde reservering
`REJECTED`	Eigenaar van de auto	Status is `REQUESTED`
`CANCELLED`	Huurder	Status is `REQUESTED` of `APPROVED` en de start is nog niet bereikt
`REQUESTED`	Niemand	Een reservering kan niet terug naar `REQUESTED` (400)

Wie niet de juiste persoon is, krijgt 403. Klopt de status of het tijdstip niet, dan volgt 409.

Voorbeelden van requests en responses

Account aanmaken: `POST /api/users/register`

```
{ "name": "Sanne de Vries", "email": "sanne@example.com", "password": "Geheim123!" }
```

Antwoord `201 Created` (zonder wachtwoord):

```
{ "id": 3, "name": "Sanne de Vries", "email": "sanne@example.com", "bonusPoints": 0 }
```

Inloggen: `POST /api/users/login`

```
{ "email": "sanne@example.com", "password": "Geheim123!" }
```

Antwoord `200 OK` :

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

Auto aanbieden: `POST /api/cars` 🔒

```
{
  "type": "BEV",
  "brand": "Tesla",
  "model": "Model 3",
  "year": 2023,
  "licensePlate": "R-123-XY",
  "seats": 5,
  "hourlyRate": 8.50,
  "kmRate": 0.20,
  "purchasePrice": 35000.0,
  "address": { "street": "Hogeschoollaan", "houseNumber": "1", "postalCode": "4818 CR",
"city": "Breda" },
  "kWhPer100Km": 17.0
}
```

Het veld `type` bepaalt welke soort auto wordt gemaakt. Bij `ICE` horen de velden `fuelType` en `litersPer100Km`, bij `BEV` het veld `kWhPer100Km` en bij `FCEV` het veld `kgPer100Km`. Ontbreekt het verbruik dat bij de soort hoort, dan volgt 400. Antwoord `201 Created` met dezelfde gegevens plus `id`, `ownerId` en `isActive: true`.

Kosten bekijken: `GET /api/cars/7/costs` 🔒 eigenaar. Antwoord `200 OK` (de BEV uit het rekenvoorbeeld in `aannames.md`):

```
{
  "costPerKm": 0.0578,
  "tcoPerYear": 8312.0,
  "kmPerYear": 15000
}
```

De kosten per km worden afgerond op vier decimalen en de TCO op twee. De opbouw van de TCO (afschrijving, verzekering, wegenbelasting, onderhoud en energie) staat in `aannames.md`; de API geeft alleen de uitkomsten terug.

Reservering aanvragen: `POST /api/reservations` 🔒

```
{ "carId": 7, "start": "2026-10-20T09:00", "end": "2026-10-20T13:00", "expectedKm": 80 }
```

Antwoord `201 Created`:

```
{
  "id": 12,
  "carId": 7,
  "renterId": 3,
  "start": "2026-10-20T09:00",
  "end": "2026-10-20T13:00",
  "expectedKm": 80,
  "estimatedPrice": 50.0,
  "status": "REQUESTED"
}
```

Aanvraag goedkeuren: `PATCH /api/reservations/12/status` 🔒 eigenaar

```
{ "status": "APPROVED" }
```

Antwoord `200 OK` met de reservering, nu met `"status": "APPROVED"`. Vraagt de huurder daarna de reservering op, dan staat er ook een veld `ownerContact` in met de naam en het e-mailadres van de eigenaar.

DTO's

De JSON-gegevens hierboven zijn in de code *DTO's* (Data Transfer Objects): eenvoudige `@Serializable` data classes in de gedeelde module `core`.

Bestand	DTO's
`UserDtos.kt`	`RegisterRequest`, `LoginRequest`, `TokenResponse`, `UserResponse`
`CarDtos.kt`	`CarRequest` (voor toevoegen én wijzigen), `CarResponse`, `AddressDto`, `CostsResponse`
`ReservationDtos.kt`	`CreateReservationRequest`, `ChangeStatusRequest`, `ReservationResponse`, `OwnerContact`
`ApiError.kt`	`ApiError` (de vorm van elke foutmelding)

De service maakt van een request-DTO een domeinobject; de route zet het domeinobject met `toResponse()` om naar een response-DTO. Zo bepaalt de API zelf welke gegevens naar buiten gaan; de wachtwoord-hash van een gebruiker komt bijvoorbeeld nooit in een response.

3.3 Foutafhandeling en statuscodes

Fouten worden in de code als *exception* gegooid. De Ktor-plugin **StatusPages** vangt ze op één centrale plek op en zet ze om naar een statuscode met een duidelijke melding. Zo werkt de voorbeeldapplicatie van de docent ook (`StatusPages.kt`). Elke foutmelding heeft dezelfde vorm, met de klasse `ApiError` uit de gedeelde module:

```
{ "message": "This car is already rented in this period." }
```

Situatie	Exception in de code	Status
Ongeldige invoer, bijvoorbeeld een negatief tarief, een eindmoment vóór het startmoment, ongeldige JSON of een id in de URL dat geen getal is	`IllegalArgumentException` (onder andere via `require()`) of `BadRequestException` (van Ktor)	**400** Bad Request
Geen geldig token, of een verkeerd e-mailadres of wachtwoord bij het inloggen	Geen exception: de authenticatie-plugin (`challenge`) of `UserRoutes` antwoordt direct	**401** Unauthorized
Ingelogd, maar niet de eigenaar of huurder	`ForbiddenException` (eigen klasse)	**403** Forbidden
Auto of reservering bestaat niet (of de auto is niet actief)	`NotFoundException` (van Ktor)	**404** Not Found
De actie kan nu niet: overlap, kenteken of e-mailadres bestaat al, verkeerde status, auto verwijderen met reserveringen in de toekomst	`IllegalStateException` via `check()` , of `ConflictException` (eigen klasse)	**409** Conflict
Te veel inlog- of registratiepogingen	Afgehandeld door de RateLimit-plugin	**429** Too Many Requests
Een onverwachte fout	Alle overige exceptions	**500** Internal Server Error, zonder technische details in het antwoord; de fout wordt wel gelogd

`require()` **en** `check()` **:** in de domeinklassen worden deze twee Kotlin-functies gebruikt (behandeld in les 3):

- `require()` controleert de **invoer**. Voorbeeld in `Reservation` : `require(end > start)` . Gaat het mis, dan volgt 400.
- `check()` controleert de **toestand** van een object. Voorbeeld in `approve()` : `check(status == REQUESTED)` . Gaat het mis, dan volgt 409.

401 of 403? Een 401 betekent "we weten niet wie je bent": de app moet de gebruiker opnieuw laten inloggen. Een 403 betekent "we weten wie je bent, maar dit mag je niet": opnieuw inloggen helpt dan niet. Dit onderscheid komt uit de authentication-les van de gastdocent.

3.4 Authenticatie en autorisatie

Authenticatie is vaststellen wie de gebruiker is. **Autorisatie** is bepalen wat die gebruiker mag.

Wachtwoorden

Wachtwoorden worden nooit leesbaar opgeslagen, maar als **hash** met het algoritme **Argon2id**:

- elk wachtwoord krijgt een eigen, willekeurige *salt*, zodat twee gelijke wachtwoorden een verschillende hash krijgen;
- de instellingen zijn dezelfde als in de demo van de gastdocent: 64 MB geheugen, 3 iteraties en een parallelliteit van 4;
- de berekening wordt gedaan met de library BouncyCastle, net als in de demo van de gastdocent.

Argon2id is de eerste keus van OWASP voor het opslaan van wachtwoorden. BCrypt wordt door OWASP alleen nog aangeraden voor oudere systemen.

Inloggen en het token

1. De gebruiker stuurt zijn e-mailadres en wachtwoord naar `POST /api/users/login`.
2. De server berekent de hash van het ingevoerde wachtwoord met de opgeslagen salt en vergelijkt die met de opgeslagen hash.
3. Klopt het, dan krijgt de gebruiker een **JWT-token** terug. Klopt het niet, dan volgt een 401 met **dezelfde foutmelding**, of nu het e-mailadres of het wachtwoord fout is. Zo kan niemand uitzoeken welke e-mailadressen een account hebben (advies van OWASP en de gastdocent).
4. Bij elk beveiligd verzoek stuurt de app het token mee in de header: `Authorization: Bearer <token>`.

Inhoud van het token:

Claim	Betekenis
`iss`	Wie het token heeft uitgegeven (de RentMyCar-server)
`aud`	Voor wie het token is bedoeld (de RentMyCar-API)
`sub`	Het id van de gebruiker
`iat`	Wanneer het token is gemaakt
`exp`	Wanneer het token verloopt

Bij elk verzoek controleert de server de handtekening en al deze claims. Is het token verlopen, aangepast of van een andere uitgever, dan volgt een 401. De inhoud van een token is voor iedereen leesbaar; daarom staan er geen wachtwoorden of andere geheime gegevens in.

De instellingen (poort, host, issuer, audience, realm en de geheime sleutel) staan in `server/resources/application.conf`, net als in de voorbeeldapplicatie van de docent. Elke instelling heeft een standaardwaarde voor de eigen computer en kan worden overschreven met een omgevingsvariabele (`${?PORT}`, `${?JWT_SECRET}` enzovoort). Zo staat er op een echte server geen geheim in de code, volgens het advies "configuration lives outside the code" uit de gastles. Bij het starten worden de instellingen ingelezen in de data classes `AppConfig`, `ServerConfig` en `JwtConfig`.

De geheime sleutel waarmee de server tokens ondertekent, komt op een echte server uit de omgevingsvariabele `JWT_SECRET`. Twee verbeteringen ten opzichte van de docent:

- De sleutel moet **minstens 32 tekens** lang zijn; `JwtConfig` controleert dat bij het starten (zoals in de auth-demo van de gastdocent). Een korte sleutel kan iemand met één token offline raden, en dan zelf tokens maken voor elke gebruiker. De standaard voor HS256 (RFC 7518) vraagt daarom minstens 256 bits.
- Is `JWT_SECRET` niet ingesteld, dan geeft de server een **waarschuwing**, omdat de standaardsleutel uit `application.conf` in git staat en dus niet geheim is.

Hoe lang is een token geldig?

Een token is **15 minuten** geldig. Een token werkt als een sleutel: wie het heeft, wordt als de gebruiker gezien. Een token kan niet worden ingetrokken; uitloggen in de app maakt een gekopieerd token niet ongeldig. Daarom is een korte geldigheid veiliger. De geldigheid staat als constante `VALID_MINUTES` in `TokenService` (de docent zet zijn 1 uur ook in de code).

Bron	Geldigheid
OWASP en andere best practices	5 tot 15 minuten, in combinatie met een refresh token
Gastdocent, authentication-les en auth-demo	15 minuten, zonder refresh token
Gastdocent, ktor-fundamentals les 9 (sprekersnotities bij de slide "The login signs the token")	"Short lifetimes plus a refresh token is the usual pattern"
Docent, voorbeeldapplicatie	1 uur

Beperking en vervolgstap: zonder refresh token moet de gebruiker na 15 minuten opnieuw inloggen. In periode 1 is dat geen probleem: de API wordt getest met `.http`-bestanden, waarin het inlogverzoek opnieuw kan worden uitgevoerd. In periode 2, wanneer de Android-app de gebruiker ingelogd moet houden, is een *refresh token* gepland:

- bij het inloggen krijgt de app naast het korte token een refresh token dat langer geldig is;
- de server bewaart de refresh tokens in de database, zodat ze bij het uitloggen kunnen worden ingetrokken;
- een refresh token mag maar één keer worden gebruikt; bij gebruik krijgt de app een nieuw token en een nieuw refresh token (*rotation*, advies van OWASP);
- de Ktor-client in de app kan dit automatisch afhandelen met `refreshTokens { }`.

Er is in het lesmateriaal en bij de (gast)docent geen uitgewerkt voorbeeld van zelfgebouwde refresh tokens; voor die uitwerking wordt de documentatie van Ktor gebruikt.

Te veel inlogpogingen

Op `POST /api/users/login` en `POST /api/users/register` zit een **rate limit** van 10 verzoeken per minuut. Daarboven geeft de server **429 Too Many Requests**. Dit maakt het raden van wachtwoorden veel lastiger. De gastdocent behandelt dit in ktor-fundamentals les 9 ("Too many logins is a 429") en gebruikt het in zijn auth-demo; de docent gebruikt het ook in zijn voorbeeldapplicatie.

Beperking: OWASP raadt aan om mislukte pogingen per account te tellen. De Ktor-plugin telt standaard per afzender. Voor deze applicatie is dat voldoende; per account tellen is een mogelijke verbetering.

Autorisatie: wie mag wat?

Er zijn geen rollen zoals "admin". Wat een gebruiker mag, hangt af van zijn relatie met een auto of reservering (zie de permissiematrix in hoofdstuk 1.6):

- de route haalt het gebruikers-id uit het token;
- de **service** controleert of deze gebruiker de eigenaar van de auto of de huurder van de reservering is;
- is dat niet zo, dan volgt een **403**.

Deze controle staat in de service en niet in de route, zodat hij met een unit test kan worden getest zonder HTTP. Dat is ook het advies van de gastdocent: "*Ownership is a check*" en "*move this check into the use case*". Het voorkomt een bekend beveiligingsprobleem: dat een ingelogde gebruiker andermans gegevens kan bekijken of wijzigen door simpelweg een ander id in de URL te zetten (OWASP API Security Top 10, nummer 1).

Herkomst van de keuzes

Onderwerp	Docent (voorbeeldapplicatie)	Gastdocent	Keuze in dit project
Wachtwoorden	Als gewone tekst opgeslagen	Argon2id	**Argon2id**, zoals de gastdocent en OWASP
Geldigheid token	1 uur	15 minuten	**15 minuten**, zoals de gastdocent en OWASP
Refresh token	Nee	Nee, genoemd als "the usual pattern"	**Gepland voor periode 2**
Instellingen	`application.conf` met omgevingsvariabelen	`application.yaml` met omgevingsvariabelen	`application.conf`, zoals de docent
Lengte geheime sleutel	Niet gecontroleerd (standaard 9 tekens)	Minstens 32 bytes	**Minstens 32 tekens**, zoals de gastdocent, plus een waarschuwing zonder `JWT_SECRET`
Rate limiting bij inloggen	Ja	Ja (5 of 10 per minuut)	**Ja, 10 per minuut**, zoals in de auth-demo
Niet de eigenaar	Vaak 404	403	**403**, zoals de gastdocent
Rollen	Rol in het token	Rollen opzoeken per verzoek	**Geen rollen**; eigenaarschap controleren in de service
Status wijzigen	`PATCH /{id}/status`	–	`PATCH /api/reservations/{id}/status`, zoals de docent
Paginering	Ja	–	**Nee**; niet nodig bij deze hoeveelheid gegevens

Bronnen:

- Vergauwen, S. *ktor-fundamentals*, les 9 "Authentication and authorization": github.com/nomisRev/ktor-fundamentals
- Vergauwen, S. *avans-toolchain-and-ktor-2026*, lessons/authentication.md en auth-demo: github.com/nomisRev/avans-toolchain-and-ktor-2026
- De Mast, P. *ktor-android-incident-demo*: github.com/pjfmast/ktor-android-incident-demo
- OWASP Password Storage Cheat Sheet: cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Authentication Cheat Sheet: cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP API Security Top 10 (2023), API1 Broken Object Level Authorization: owasp.org/API-Security
- Ktor documentatie, Rate limiting: ktor.io/docs/server-rate-limit.html

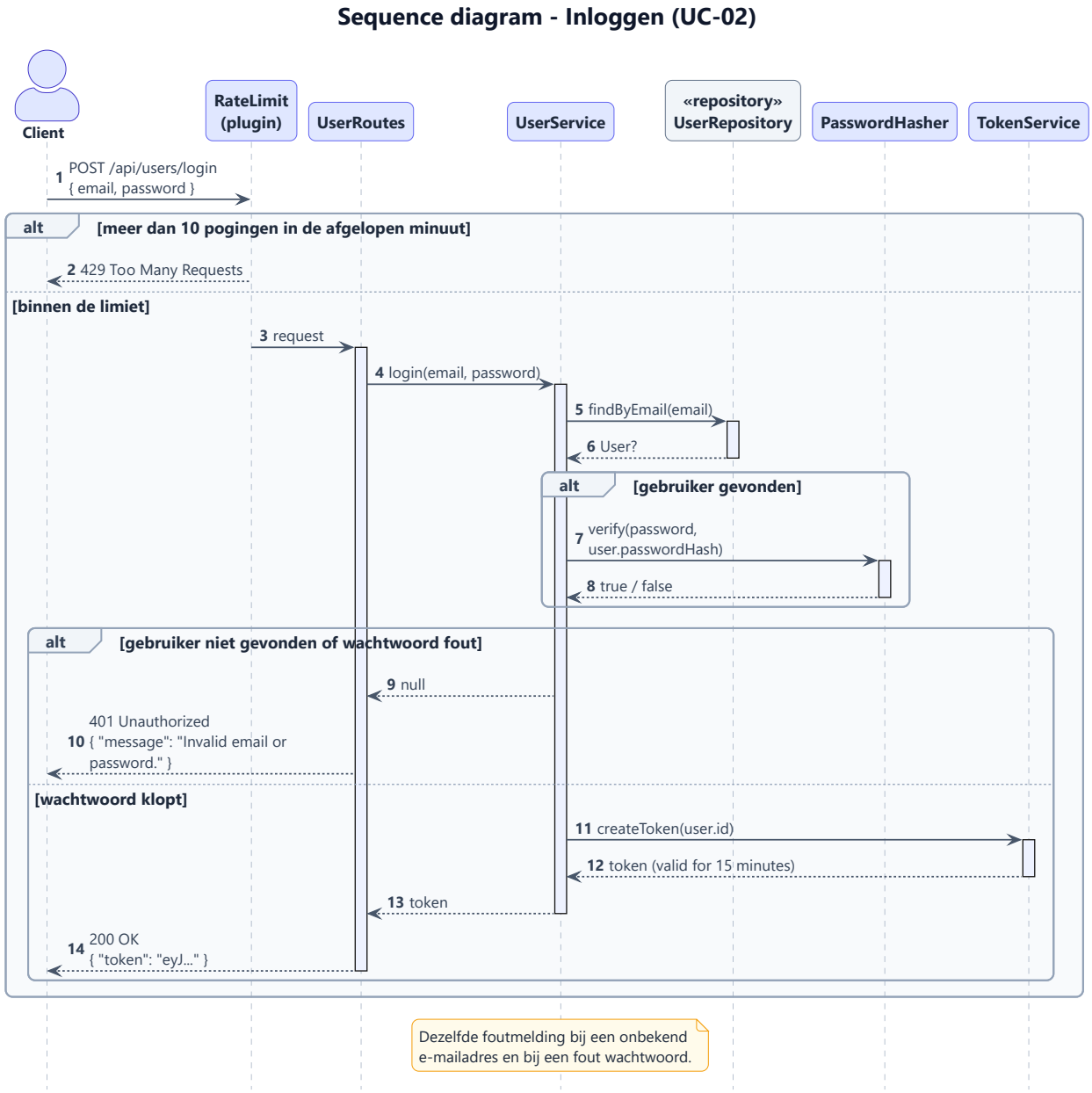
3.5 AI-agent

Nog niet gerealiseerd. De AI-agent (FE-20, UC-14) wordt later uitgewerkt; het taalmodel draait dan lokaal en is gratis (NFE-09).

3.6 Sequence diagrams

De sequence diagrams laten zien hoe de onderdelen uit 3.1 samenwerken bij één verzoek. Elk diagram toont zowel het pad waarin alles goed gaat als de paden waarin iets misgaat (de `alt`-blokken). De kleuren volgen de lagen: blauw voor plugins, routes en services, grijs voor repositories en groen voor domeinobjecten.

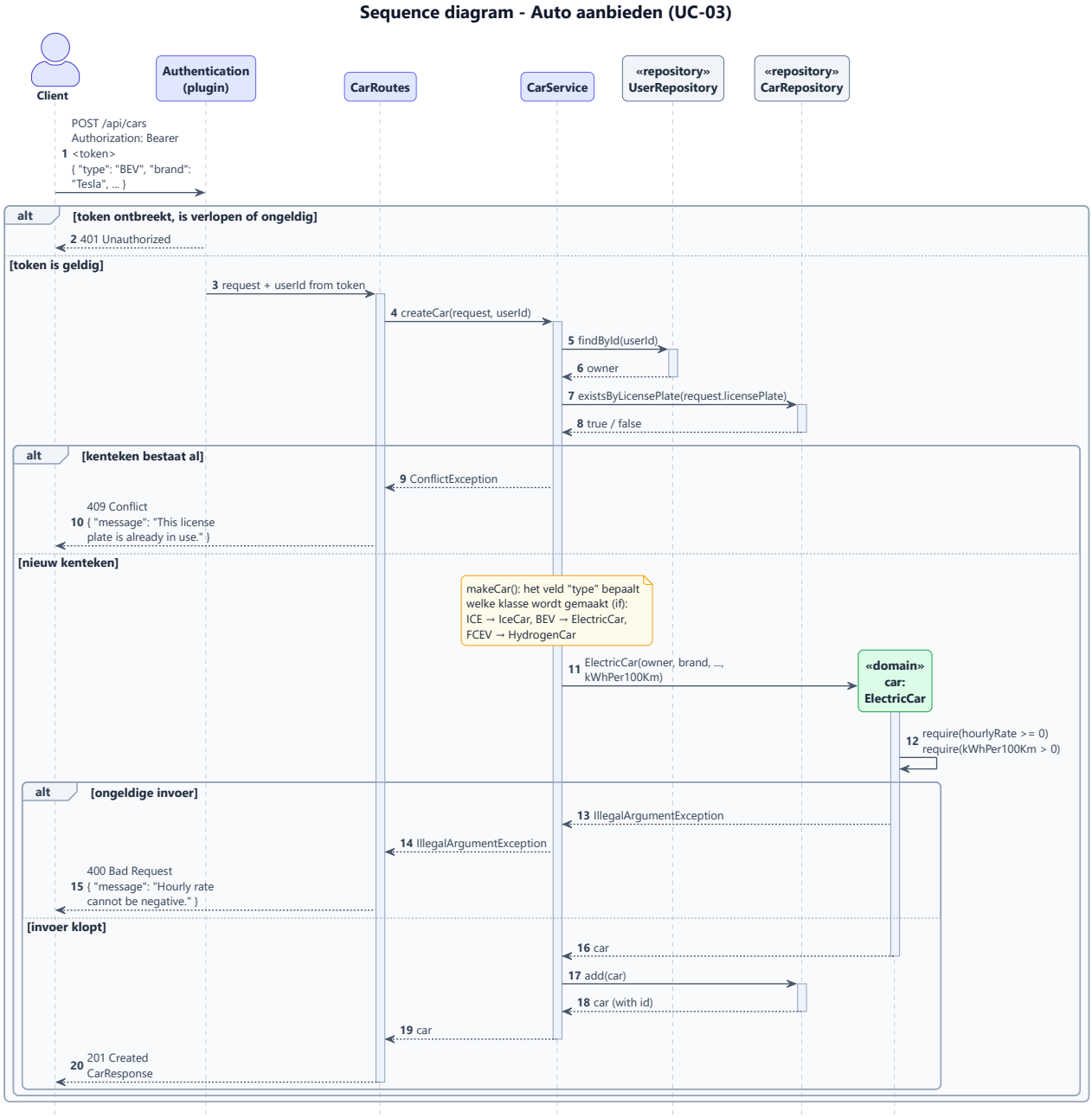
Inloggen (UC-02)



Sequence diagram inloggen
Bron: `puml/sequence-login.puml`

- De **RateLimit**-plugin weigert het verzoek met 429 als er in de afgelopen minuut al 10 pogingen zijn geweest.
- `UserService` zoekt de gebruiker op e-mailadres en laat `PasswordHasher` het wachtwoord controleren tegen de opgeslagen Argon2id-hash.
- Een onbekend e-mailadres en een fout wachtwoord geven **dezelfde** 401-melding.
- Klopt alles, dan maakt `TokenService` een token dat 15 minuten geldig is.

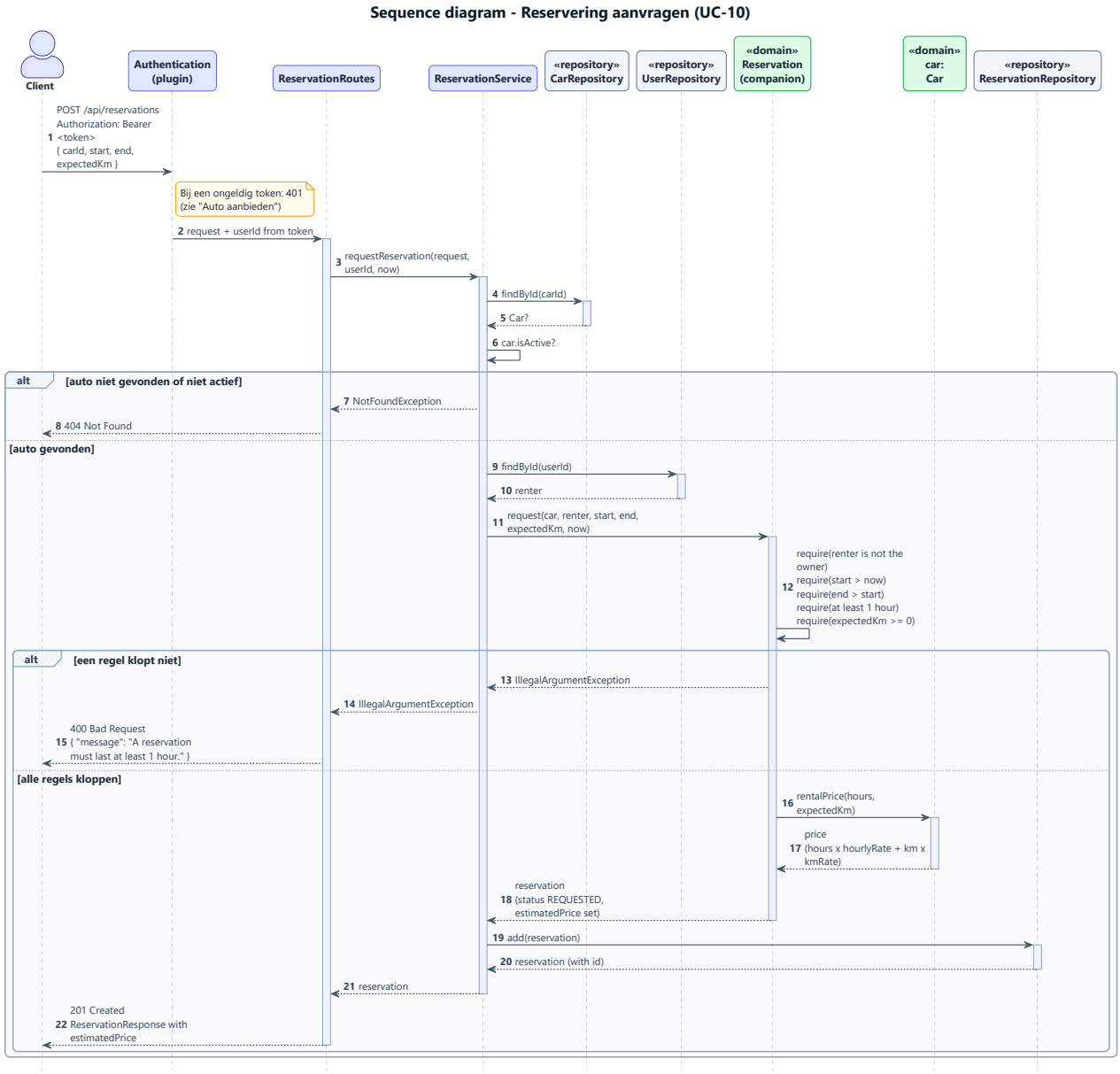
Auto aanbieden (UC-03)



Sequence diagram auto aanbieden
Bron: `puml/sequence-create-car.puml`

- De **Authentication**-plugin controleert het token. Bij een ontbrekend, verlopen of ongeldig token volgt 401; anders krijgt de route het gebruikers-id.
- `CarService` haalt de eigenaar op en controleert dat het kenteken nog niet bestaat (anders 409).
- In `makeCar()` bepaalt het veld `type` welke klasse wordt gemaakt: in dit voorbeeld een `ElectricCar`. Ontbreekt het verbruik van die soort, dan volgt 400. Bij het maken controleert de klasse zelf de invoer met `require()` (anders 400).
- De repository slaat de auto op en geeft hem terug met een id; de route antwoordt met 201.

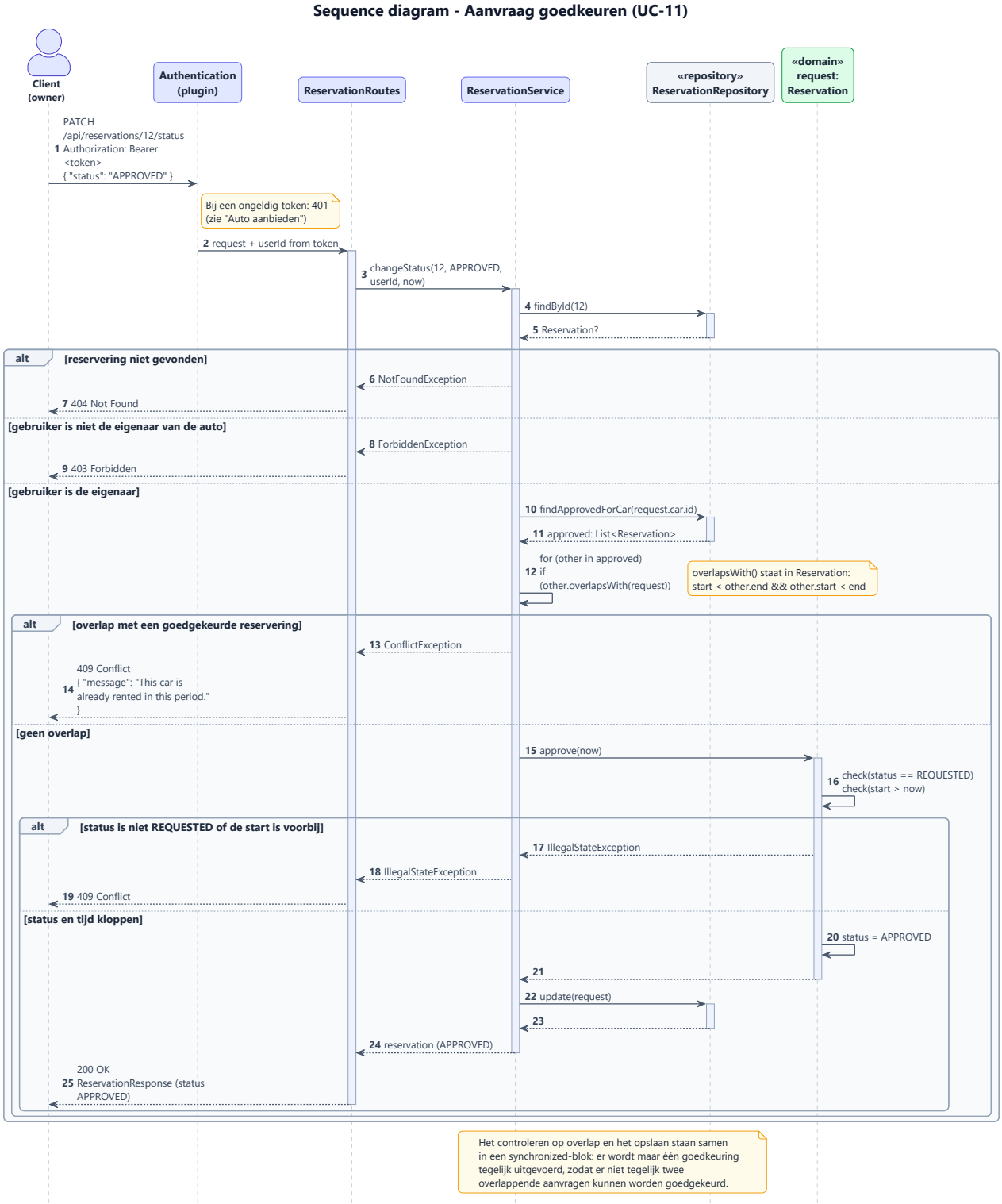
Reservering aanvragen (UC-10)



Sequence diagram reservering aanvragen
Bron: `puml/sequence-request-reservation.puml`

- `ReservationService` zoekt de auto op. Een auto die niet bestaat of niet actief is, geeft 404.
- De fabrieksfunctie `Reservation.request()` controleert alle regels voor een reservering met `require()` (anders 400).
- De geschatte prijs wordt berekend door de auto zelf (`car.rentalPrice()`), omdat de tarieven bij de auto horen. De prijs wordt vastgelegd in de reservering.
- De reservering wordt opgeslagen met status `REQUESTED` .

Aanvraag goedkeuren (UC-11)



Sequence diagram aanvraag goedkeuren

Bron: `puml/sequence-approve-reservation.puml`

- `ReservationService` zoekt de aanvraag op (anders 404) en controleert of de ingelogde gebruiker de eigenaar van de auto is (anders 403).
- De service haalt de goedgekeurde reserveringen van dezelfde auto op en controleert met `overlapsWith()` of er een overlapt (anders 409). Dit is de regel uit FE-15: een eigenaar kan een auto niet twee keer voor dezelfde periode verhuren.
- `approve(now)` controleert met `check()` dat de status nog `REQUESTED` is en dat de start nog niet voorbij is (anders 409), en zet daarna de status op `APPROVED`.
- **Gelijktijdige verzoeken:** het controleren op overlap en het opslaan staan samen in een `synchronized`-blok. Daardoor wordt er maar één goedkeuring tegelijk uitgevoerd. Zonder dit blok zouden twee goedkeuringen die op precies hetzelfde moment binnenkomen allebei "geen overlap" kunnen zien. *Beperking:* dit werkt omdat er één server draait; met meerdere servers naast elkaar zou een databasetransactie met een lock nodig zijn.

4. Testen

In dit hoofdstuk staat hoe de Web API wordt getest: welke soorten tests er zijn, welke tests gepland zijn en wat de uitkomsten zijn.

4.1 Teststrategie

De testpiramide

De tests volgen de testpiramide uit les 3: veel kleine, snelle tests onderin en een paar grotere tests bovenin.

```
        ▲      .http-flows        handmatig, ook gebruikt in de demo
       ▲▲▲     Integratietests    database en HTTP, met testApplication
      ▲▲▲▲▲▲   Unit tests         domein en services, zonder database of server
```

Niveau	Wat wordt getest	Hoe	Snelheid
Unit tests	De regels en berekeningen in het domein, en de services	Zonder database en zonder server; services krijgen een `FakeRepository`	Zeer snel
Integratietests	De samenwerking tussen onderdelen: repository en database, routes en services	Echte Exposed-repositories met een H2-database in het geheugen; echte routes met `testApplication`	Snel
`.http`-flows	Complete routes door de API, zoals een gebruiker ze doorloopt	Handmatig in IntelliJ, tegen de draaiende server	Handmatig

Unit tests

Unit tests controleren één klasse of functie los van de rest:

- **domein:** de kosten per km en de TCO per soort auto, de huurprijs, de regels voor een reservering, `overlapsWith()` en de statusovergangen;
- **services:** de regels waarvoor gegevens uit een repository nodig zijn, zoals "alleen de eigenaar mag goedkeuren". De service krijgt daarvoor een `FakeRepository`: een eenvoudige versie die gegevens in een lijst bewaart. Dit kan omdat de service alleen de interface van de repository kent (zie hoofdstuk 3.1);
- **beveiliging:** het hashen en controleren van wachtwoorden en het afwijzen van verlopen tokens (gepland, zie 4.2).

De rekenvoorbeelden uit `aannames.md` worden gebruikt als verwachte uitkomsten.

Integratietests

Integratietests controleren of onderdelen goed samenwerken:

- **Database:** de echte Exposed-repositories worden getest tegen een H2-database in het geheugen. Zo wordt onder andere gecontroleerd of de vertaling van het OO-model naar de tabellen klopt (hoofdstuk 2.5): een `ElectricCar` die wordt opgeslagen, moet er ook weer als `ElectricCar` uitkomen. Zo test de docent zijn repositories ook (`ExposedRepositoriesTest.kt` in de voorbeeldapplicatie).
- **HTTP:** de echte routes, plugins en services worden getest met `testApplication` van Ktor. De server en een testclient draaien dan in hetzelfde proces, zonder dat er een poort open hoeft. De client stuurt echte HTTP-verzoeken met JSON en controleert de statuscode en de inhoud van het antwoord. Dit is de werkwijze uit les 7 en uit les 8 van de gastdocent ("*A test runs server and client in one process*").

`.http` -flows

Per soort gebruiker is er een `.http` -bestand met een complete route door de API, zoals in de voorbeeldapplicatie van de docent:

Bestand	Route
`visitor-flow.http`	Auto's zoeken en bekijken zonder in te loggen
`owner-flow.http`	Registreren → inloggen → auto aanbieden → auto wijzigen → kosten bekijken → aanvragen bekijken → aanvraag goedkeuren → auto verwijderen
`renter-flow.http`	Inloggen → auto zoeken → reservering aanvragen → reservering bekijken → annuleren

Foto uploaden komt in periode 2 bij `owner-flow.http` . IntelliJ bewaart het token na het inloggen automatisch voor de volgende verzoeken (`client.global.set(...)`). Deze bestanden worden gebruikt om de API bij het assessment te demonstreren en om het testrapport (4.4) in te vullen. Ze staan in `server/test/http-requests/` .

Reproduceerbare tests

Leeruitkomst 1 vraagt om reproduceerbare tests: een test moet elke keer dezelfde uitkomst geven.

- **Lege database per test:** elke integratietest begint met een nieuwe, lege H2-database in het geheugen, met een eigen naam (`System.nanoTime()`). Tests beïnvloeden elkaar dus niet.
- **Vaste tijd:** functies die van de tijd afhangen, krijgen de huidige tijd mee als parameter (`now` , zie hoofdstuk 2.1). In een test wordt een vaste tijd gebruikt, zodat een test over "de start ligt in het verleden" altijd hetzelfde resultaat geeft.
- **Vaste testgegevens:** de tests gebruiken vaste gebruikers, auto's en tijden uit `TestData.kt` (bijvoorbeeld `OWNER` , `RENTER` , `petrolCar()` en `NOW`). De fake repositories staan in `Fakes.kt` .
- **Opbouw van een test:** elke test volgt *Arrange, Act, Assert*: eerst klaarzetten, dan de actie uitvoeren, dan de uitkomst controleren.

Gereedschap

Gereedschap	Waarvoor
`kotlin.test`	Het testframework voor alle tests. Dit is de standaard in een Kotlin Toolchain-project en wordt gebruikt in les 3 en in de voorbeeldapplicatie van de docent.
`ktor-server-test-host`	`testApplication` voor de HTTP-tests (staat al in het project).
H2 (in het geheugen)	De database voor de integratietests.
IntelliJ HTTP Client	Het uitvoeren van de `.http`-flows.

De tests van de server worden uitgevoerd met `./kotlin test -m server` (of `.\kotlin.bat test -m server` op Windows). Zonder `-m server` worden ook de modules van de Android-app gebouwd; dat geeft nu nog een interne build-fout en wordt in periode 2 opgelost.

Afwijkingen van de les

Onderwerp	Les / (gast)docent	Keuze	Waarom
Testbibliotheek	De gastdocent gebruikt Kotest (`shouldBe`)	`kotlin.test`	Standaard in de Kotlin Toolchain en gebruikt in les 3 en door de docent; één bibliotheek minder om te leren.
Database in tests	Leeruitkomst 1 noemt Testcontainers als mogelijkheid	H2 in het geheugen	Testcontainers heeft Docker nodig. H2 is ook de database van de applicatie zelf, en de docent gebruikt het ook in zijn tests.
Dekkingspercentage	–	Niet gemeten	De tests richten zich op de belangrijke regels en flows; een percentage zegt weinig over de kwaliteit van de tests.

4.2 Unit tests

✱ = deze tests worden als eerste gemaakt; samen voldoen ze aan de minimale eis van drie unit tests. De overige tests maken de controle vollediger.

De kolom *Gebouwd* geeft aan of de test al in de code staat (in `server/test/`). De tests die nog niet zijn gebouwd, staan gepland; het gedrag dat ze controleren is wel met de hand getest (zie 4.4).

Nr	Test	Verwachte uitkomst	Eis	Gebouwd
UT-01 ✱	Kosten per km van een benzineauto met een verbruik van 6 liter per 100 km	€ 0,1452	FE-08	✅ `CarCostsTest`
UT-02	Kosten per km van een BEV (17 kWh per 100 km) en van een FCEV (1 kg per 100 km)	€ 0,0578 en € 0,21	FE-08	Gepland
UT-03 ✱	TCO van een benzineauto van € 30.000 (rekenvoorbeeld uit `aannames.md`)	€ 9.128 per jaar	FE-09	✅ `CarCostsTest`
UT-04	Huurprijs van 4 uur met een uurtarief van € 6,00 en 80 km met een km-tarief van € 0,25	€ 44,00	FE-13	Gepland

Nr	Test	Verwachte uitkomst	Eis	Gebouwd
UT-05 ✱	`overlapsWith()` voor vier gevallen: 9–12 en 11–14 (ja), 9–12 en 12–15 (nee, sluit aan), 9–12 en 13–15 (nee), 9–17 en 10–11 (ja, valt erbinnen)	ja, nee, nee, ja	FE-15	✅ `ReservationTest`
UT-06	`Reservation.request()` met een eindmoment vóór het startmoment, een duur korter dan 1 uur, een startmoment in het verleden, en de eigen auto	Vier keer een `IllegalArgumentException`	FE-12	Gepland
UT-07	`approve()` op een reservering met status `REJECTED` of `CANCELLED`	`IllegalStateException`	FE-14	Gepland
UT-08	`cancel()` nadat het startmoment is bereikt	`IllegalStateException`	FE-16	Gepland
UT-09 ✱	`ReservationService` : een aanvraag goedkeuren terwijl er al een overlappende goedgekeurde reservering is	`ConflictException` ; de status blijft `REQUESTED`	FE-15	✅ `ServiceTest`
UT-10	`ReservationService` : een gebruiker die niet de eigenaar is, probeert een aanvraag goed te keuren	`ForbiddenException`	Permissiematrix (1.6)	Gepland
UT-11	`CarService` : een auto verwijderen die nog een goedgekeurde reservering in de toekomst heeft	`ConflictException` ; de auto blijft actief	FE-06	Gepland
UT-12	`PasswordHasher` : een wachtwoord hashen en daarna controleren met het juiste en met een fout wachtwoord	`true` en `false`	NFE-05	Gepland
UT-13	`TokenService` : een token waarvan de geldigheid is verlopen, wordt afgewezen	Token is ongeldig	NFE-04	Gepland

4.3 Integratietests

✱ = deze tests worden als eerste gemaakt; samen voldoen ze aan de minimale eisen van drie integratietests en tests voor de vier soorten CRUD-requests.

Nr	Test	Verwachte uitkomst	Eis	Gebouwd
IT-01 ✱	`ExposedCarRepository` : een `IceCar` , een `ElectricCar` en een `HydrogenCar` opslaan en weer ophalen	Dezelfde soort en dezelfde gegevens terug	2.5	✅ `CarRepositoryTest`

Nr	Test	Verwachte uitkomst	Eis	Gebouwd
IT-02	`ExposedCarRepository` : een auto op niet actief zetten en daarna zoeken	De auto komt niet meer terug	FE-06	Gepland
IT-03 ✱	CRUD op auto's: `POST /api/cars` , `GET /api/cars/{id}` , `PUT /api/cars/{id}` , `DELETE /api/cars/{id}` , daarna opnieuw `GET`	201, 200, 200, 204, 404	FE-03 t/m FE-06	✅ `ApiTest`
IT-04 ✱	`POST /api/cars` zonder token	401	NFE-04	✅ `ApiTest`
IT-05	`PUT /api/cars/{id}` voor de auto van een andere gebruiker	403	Permissiematrix (1.6)	Gepland
IT-06	`GET /api/cars/{id}` voor een auto die niet bestaat	404	FE-11	Gepland
IT-07	Registreren → inloggen → `GET /api/users/me`	201, 200, 200	FE-01, FE-02	Gepland
IT-08	Inloggen met een fout wachtwoord	401 met de algemene foutmelding	FE-02	Gepland
IT-09	Reservering aanvragen → goedkeuren → een tweede, overlappende aanvraag goedkeuren	201, 200, 409	FE-12, FE-14, FE-15	Gepland
IT-10	`GET /api/cars?type=BEV&city=Breda`	Alleen actieve BEV's in Breda	FE-10	Gepland
IT-11	Elf keer inloggen binnen een minuut	De elfde keer 429	NFE-10	Gepland

Stand: er zijn 7 geautomatiseerde tests gebouwd: 4 unit tests en 3 integratietests. Daarmee is voldaan aan de minimale eis van drie unit tests, drie integratietests en tests voor de vier soorten CRUD-requests (NFE-07). Alle 7 tests slagen (`.\kotlin.bat test -m server` , 25 oktober 2026).

4.4 Testrapport API-requests

Dit testrapport is ingevuld op 25 oktober 2026. De requests zijn uitgevoerd tegen de draaiende server (`.\kotlin.bat run -m server`) met de demodata en met de verzoeken uit de `.http` -bestanden. Een paar situaties worden ook door een geautomatiseerde test gecontroleerd; dat staat erbij.

Request	Situatie	Verwacht	Werkelijk	Resultaat
`POST /api/users/register`	Nieuw account	201	201	✅
`POST /api/users/register`	E-mailadres bestaat al	409	409	✅
`POST /api/users/login`	Juist wachtwoord	200 + token	200 + token	✅
`POST /api/users/login`	Fout wachtwoord	401	401, "Invalid email or password."	✅

Request	Situatie	Verwacht	Werkelijk	Resultaat
`POST /api/users/login`	Meer dan 10 pogingen per minuut	429	De elfde poging 429	✅
`GET /api/users/me`	Met geldig token	200	200	✅
`GET /api/users/me`	Zonder token	401	401	✅
`GET /api/cars`	Zonder filters	200 + alle actieve auto's	200 + alle actieve auto's	✅
`GET /api/cars?type=…&city=…`	Met filters	200 + passende auto's	200, alleen BEV's in Breda	✅
`GET /api/cars?type=…`	Onbekende soort auto	400	400	✅
`GET /api/cars?maxHourlyRate=…`	Uurprijs is geen getal	400	400	✅
`GET /api/cars?from=…&to=…`	Einde vóór begin	400	–	Periode 2
`GET /api/cars/{id}`	Bestaande auto	200	200 (ook in IT-03)	✅
`GET /api/cars/{id}`	Onbekende of niet-actieve auto	404	404 (ook in IT-03)	✅
`GET /api/cars/mine`	Met geldig token	200 + eigen auto's	200 + eigen auto's	✅
`POST /api/cars`	Geldige auto	201	201 (ook in IT-03)	✅
`POST /api/cars`	Negatief tarief	400	400	✅
`POST /api/cars`	Kenteken bestaat al	409	409	✅
`POST /api/cars`	Zonder token	401	401 (IT-04)	✅
`PUT /api/cars/{id}`	Eigen auto	200	200 (ook in IT-03)	✅
`PUT /api/cars/{id}`	Auto van een ander	403	403	✅
`DELETE /api/cars/{id}`	Eigen auto zonder toekomstige reserveringen	204	204 (ook in IT-03)	✅
`DELETE /api/cars/{id}`	Auto met een toekomstige reservering	409	409	✅
`DELETE /api/cars/{id}`	Auto van een ander	403	403	✅
`GET /api/cars/{id}/costs`	Eigen auto	200 + kosten	200 + kosten	✅
`GET /api/cars/{id}/costs`	Auto van een ander	403	403	✅
`POST /api/cars/{id}/photos`	Geldige afbeelding	201	–	Periode 2

Request	Situatie	Verwacht	Werkelijk	Resultaat
`POST /api/cars/{id}/photos`	Ongeldig bestandstype	400	–	Periode 2
`DELETE /api/cars/{id}/photos/{photoId}`	Eigen foto	204	–	Periode 2
`GET /api/photos/{fileName}`	Bestaande foto	200 + afbeelding	–	Periode 2
`POST /api/reservations`	Geldige aanvraag	201 + geschatte prijs	201 + geschatte prijs	✅
`POST /api/reservations`	Korter dan 1 uur	400	400	✅
`POST /api/reservations`	Eigen auto	400	400, "You cannot reserve your own car."	✅
`GET /api/reservations/mine`	Met geldig token	200 + eigen reserveringen	200 + eigen reserveringen	✅
`GET /api/cars/{id}/reservations`	Eigen auto	200	200	✅
`GET /api/reservations/{id}`	Na goedkeuring, als huurder	200 + contactgegevens eigenaar	200 + `ownerContact`	✅
`GET /api/reservations/{id}`	Reservering van een ander	403	403	✅
`PATCH /api/reservations/{id}/status`	Goedkeuren door de eigenaar	200	200	✅
`PATCH /api/reservations/{id}/status`	Goedkeuren met overlap	409	409 (ook in UT-09)	✅
`PATCH /api/reservations/{id}/status`	Goedkeuren door een ander dan de eigenaar	403	403	✅
`PATCH /api/reservations/{id}/status`	Annuleren door de huurder vóór de start	200	200	✅
`PATCH /api/reservations/{id}/status`	Annuleren na de start	409	409, "This reservation has already started."	✅

Wat de tests opleverden

Wordt ingevuld na het bouwen: welke fouten de tests hebben gevonden en wat er daardoor in het ontwerp of de code is verbeterd.

5. Installatie en gebruik

5.1 Vereisten

Wat	Toelichting
Windows, macOS of Linux	De applicatie draait lokaal op de eigen computer (NFE-08).
Internetverbinding bij de eerste start	De scripts `kotlin.bat` (Windows) en `kotlin` (macOS/Linux) in de hoofdmap downloaden bij de eerste keer zelf de Kotlin Toolchain en een Java-runtime. Een aparte installatie van Java of Gradle is niet nodig.
IntelliJ IDEA (aanbevolen)	Voor het lezen van de code en het uitvoeren van de `.http`-bestanden met de ingebouwde HTTP Client.

Er is geen aparte databaseserver nodig: de H2-database draait mee in de applicatie.

5.2 Project starten

Open een terminal in de hoofdmap van het project.

Actie	Windows	macOS/Linux
Server starten	`.\kotlin.bat run -m server`	`./kotlin run -m server`
Tests van de server uitvoeren	`.\kotlin.bat test -m server`	`./kotlin test -m server`

De server draait daarna op http://localhost:8080. Alle endpoints beginnen met `/api` (zie hoofdstuk 3.2). Stoppen gaat met `Ctrl+C`.

Instellingen. De instellingen staan in `server/resources/application.conf`. Elke instelling kan worden overschreven met een omgevingsvariabele:

Omgevingsvariabele	Betekenis	Standaard
`PORT`	Poort van de server	`8080`
`HOST`	Adres waarop de server luistert	`0.0.0.0`
`JWT_SECRET`	Geheime sleutel voor de tokens, minstens 32 tekens	Een sleutel voor de eigen computer
`JWT_ISSUER`	Uitgever van de tokens	`http://localhost:8080`
`JWT_AUDIENCE`	Voor wie de tokens bedoeld zijn	`rentmycar-api`

Zonder `JWT_SECRET` start de server met de sleutel uit `application.conf` en toont hij een waarschuwing. Dat is goed genoeg voor de eigen computer, maar niet voor een echte server, omdat die sleutel in git staat (zie hoofdstuk 3.4). Een eigen sleutel instellen kan zo (Windows PowerShell):

```
$env:JWT_SECRET = "een-lange-geheime-sleutel-van-minstens-32-tekens"
.\kotlin.bat run -m server
```

Database. De gegevens worden opgeslagen in de map `data/` (`data/rentmycar.mv.db`) en blijven bewaard na een herstart. Deze map staat niet in git. Wie opnieuw wil beginnen met alleen de testdata, stopt de server, verwijdert de map `data/` en start de server opnieuw.

5.3 Testdata en testaccounts

Bij de start controleert de server of de database leeg is. Is dat zo, dan vult `DemoData.kt` de database met realistische testdata (NFE-02).

Testaccounts (wachtwoord voor alle accounts: `Welkom123!`):

Naam	E-mailadres	Rol in de testdata
Eva de Vries	`eva@example.com`	Eigenaar van de Volkswagen, de Peugeot en de Tesla
Sanne Jansen	`sanne@example.com`	Eigenaar van de Dacia, de Kia, de Renault en de Toyota
Daan Bakker	`daan@example.com`	Huurder met twee reserveringen

Auto's:

Id	Auto	Soort	Plaats	Uurtarief	Eigenaar
1	Volkswagen Golf	ICE (benzine)	Breda	€ 6,00	Eva
2	Peugeot 308 SW	ICE (diesel)	Breda	€ 5,50	Eva
3	Dacia Jogger	ICE (LPG)	Tilburg	€ 5,00	Sanne
4	Tesla Model 3	BEV	Breda	€ 8,50	Eva
5	Kia EV6	BEV	Tilburg	€ 9,00	Sanne
6	Renault Zoe	BEV	Den Bosch	€ 4,50	Sanne
7	Toyota Mirai	FCEV	Den Bosch	€ 10,00	Sanne

Reserveringen van Daan (de tijden worden berekend vanaf het moment dat de testdata wordt aangemaakt):

- de Volkswagen Golf over twee dagen, 4 uur, status `APPROVED`;
- de Tesla over drie dagen, 3 uur, status `REQUESTED`.

5.4 De API gebruiken

De API wordt getest en gedemonstreerd met drie `.http`-bestanden in `server/test/http-requests/`:

Bestand	Inhoud
`visitor-flow.http`	Auto's zoeken en bekijken zonder in te loggen
`owner-flow.http`	Als nieuwe eigenaar registreren, inloggen, een auto aanbieden, wijzigen, de kosten bekijken, een aanvraag goedkeuren en de auto verwijderen
`renter-flow.http`	Als Daan inloggen, een auto zoeken, een reservering aanvragen, bekijken en annuleren

Werkwijze in IntelliJ:

1. Start de server (zie 5.2).
2. Open een `.http`-bestand en klik op het groene pijltje naast een verzoek.
3. Voer eerst het inlogverzoek uit. IntelliJ bewaart het token automatisch en gebruikt het bij de volgende verzoeken.
4. Een token is 15 minuten geldig. Krijg je daarna een 401, voer dan het inlogverzoek opnieuw uit.

Let op:

- Inloggen en registreren mogen maximaal 10 keer per minuut. Daarna geeft de server 429; wacht dan een minuut.
- `owner-flow.http` registreert `fleur@example.com` met kenteken `V-111-FS`. Bij een tweede keer uitvoeren bestaan die al (409). Verwijder dan de map `data/` of pas het e-mailadres en het kenteken aan.

Voorbeeld met curl (zonder IntelliJ):

```
# Alle elektrische auto's in Breda
curl "http://localhost:8080/api/cars?type=BEV&city=Breda"

# Inloggen als Daan
curl -X POST http://localhost:8080/api/users/login -H "Content-Type: application/json" -d "
{\"email\":\"daan@example.com\",\"password\":\"Welkom123!\"}"
```

6. Productverantwoording

Dit hoofdstuk is met hulp van AI opgesteld op basis van mijn AI-log en het verloop van het project.

6.1 Verantwoording AI-gebruik

Ik heb tijdens het hele project gewerkt met **Claude Code** (model Claude Opus 5.5), in de terminal naast IntelliJ IDEA. De uitgebreide verantwoording, per datum met wat ik vroeg, wat de AI voorstelde en wat ik besloot, staat in het losse document **AI-log**. Hieronder de samenvatting.

Waarvoor ik AI heb gebruikt:

Fase	Wat de AI deed	Wat ik deed
Analyse	De studiedocumenten samengevat, vragen gesteld over de casus en voorstellen gedaan	De scope en de regels van de casus bepaald, en voorstellen afgewezen die niet bij de casus pasten
Ontwerp	Voorstellen voor het domeinmodel, de architectuur en de database; de diagrammen in PlantUML gemaakt	Per onderdeel gevraagd of het uit het lesmateriaal kwam, en de keuzes gemaakt
Realisatie	Uitleg van Kotlin-begrippen, voorbeelden van code gegeven	De stijl van de code bepaald, een deel zelf geschreven, en voorstellen beoordeeld
Testen	Een teststrategie voorgesteld, tests geschreven en de API met de hand getest	Gecontroleerd of de verwachte uitkomsten klopten, met de rekenvoorbeelden
Documentatie	Hoofdstukken uitgeschreven en de documentatie gelijkgemaakt aan de software	De inhoud bepaald en eerlijk aangegeven wat nog niet gerealiseerd is

Hoe ik de AI heb gecontroleerd:

- **Bronnen vragen.** De AI geeft bij elke keuze aan of die uit het lesmateriaal komt, van de docent of de gastdocent, of een eigen voorstel is. Zo kwam bijvoorbeeld uit dat de kluiscode voor de sleutel niet in de casus stond.
- **Vergelijken met de docent.** Ik liet de AI de code van de docent en de gastdocent bekijken, en koos bewust voor de aanpak van de docent met mijn eigen verbeteringen (`application.conf`, met tokens van 15 minuten en een sleutel van minstens 32 tekens).
- **Zelf twijfelen.** Bij de beveiliging stelde de AI eerst tokens van 24 uur, BCrypt en geen rate limiting voor. Mijn twijfel daarover klopte: na extra onderzoek (OWASP, gastles) werden het 15 minuten, Argon2id en rate limiting.
- **Testen.** Na elke wijziging draaiden de tests; aan het einde is de hele API met de hand getest (testrapport, hoofdstuk 4.4).

Waar ik een voorstel van de AI niet heb overgenomen: GPS-coördinaten voor de plek van een auto, de kluiscode, het meeverwijderen van reserveringen (ik koos *soft delete*), compacte Kotlin-code die ik slecht kon lezen, en een uitgebreid sjabloon voor dit document dat bedoeld is voor een groter team.

6.2 Highlights

- **Het domeinmodel.** `Car` is een sealed class met drie soorten auto's. De TCO staat één keer in `Car` en werkt voor elke soort, doordat elke soort zijn eigen `costPerKm()` en `fixedCostsPerYear()` heeft (polymorfisme).
- **Onderbouwde beveiliging.** Argon2id, tokens van 15 minuten, rate limiting en eigenaarschap controleren in de service (403). Voor elke keuze is de bron bekend: OWASP, de docent of de gastdocent.
- **Leesbare code.** Functies met `return` onderaan, geen `it`, `for`-lussen in plaats van `.map {}`, en boven elke databasequery de SQL in het commentaar. Zo kan ik elke regel uitleggen.
- **Alles getest.** Alle 7 geautomatiseerde tests slagen, en alle situaties uit het testrapport geven de verwachte statuscode.
- **Eerlijke documentatie.** Wat niet is gebouwd, staat er ook zo in, met de planning voor periode 2.

6.3 Lowlights en verbeterpunten

- **Niet alles is gebouwd.** Foto's en zoeken op beschikbaarheid in een periode volgen in periode 2. De AI-agent is nog niet gerealiseerd.
- **Minder geautomatiseerde tests dan gepland.** Er zijn 7 van de 24 geplande tests gebouwd. Dat is genoeg voor de minimale eis, maar het gedrag van de andere tests is alleen met de hand gecontroleerd.
- **Git ging een keer mis.** Door een vastgelopen merge moesten de branches in één keer worden opgeruimd. Vaker en kleiner committen voorkomt dat.
- **Verbeterpunten in de code:**
  - het kenteken gelijkmaken (nu zijn `ab-123-c` en `AB-123-C` twee verschillende auto's);
  - het adres en het bouwjaar controleren, en een bewuste keuze maken over een tarief van € 0;
  - de eigenaar van een auto wordt per auto apart opgehaald (het N+1-probleem); bij veel auto's is een join beter;
  - goedkeuren is beveiligd met `synchronized`, wat alleen werkt met één server;
  - refresh tokens, zodat de app in periode 2 ingelogd kan blijven.

6.4 Lessons learned en advies

Wat ik heb geleerd:

- AI-voorstellen klinken logisch, maar ik moet zelf nagaan of ze kloppen met de casus en met de praktijk.
- Bij beveiliging neem ik niet het eerste antwoord over, maar vraag ik naar de bronnen.
- Een AI-voorstel vergelijken met hoe de docent het doet, geeft een betere en beter te onderbouwen keuze.
- Een AI maakt kleine fouten die je pas ziet als je het resultaat zelf goed bekijkt.
- Ik beslis zelf wat ik overneem, en kies voor code die ik kan lezen en uitleggen.

Advies aan een volgende student:

- Maak vooraf afspraken met de AI, bijvoorbeeld in een `CLAUDE.md`: eerst overleggen, eenvoudige code, en altijd de bron noemen.
- Laat de AI niet te veel tegelijk schrijven. Kleine stappen zijn beter te begrijpen en te controleren.
- Commit vaak en klein, zodat je makkelijk terug kunt als er iets misgaat.

7. Android app

Wordt uitgewerkt in periode 2.

Bijlage A – Aannames

In dit document staan de vaste waarden en formules die de applicatie gebruikt voor de berekeningen. Prijzen veranderen; de waarden hieronder gelden op de peildatum. In de code staan ze op één plek, in het object `CostAssumptions` (zie hoofdstuk 2), zodat ze makkelijk kunnen worden aangepast.

Peildatum: 5 oktober 2026.

Formules

Kosten per kilometer (gebruikt bij FE-08 en de TCO)

> kosten per km = verbruik per 100 km × energieprijs ÷ 100

De formule is voor alle soorten auto's gelijk, maar elke soort heeft een eigen verbruik en energieprijs. Het verbruik vult de eigenaar in; de energieprijzen staan hieronder.

TCO per jaar (FE-09)

> TCO = afschrijving + verzekering + wegenbelasting + onderhoud + energiekosten
> afschrijving = aanschafprijs × 15%
> energiekosten = 15.000 km × kosten per km

Geschatte huurprijs (FE-13)

> huurprijs = uurtarief × aantal uren + km-tarief × verwachte kilometers

Energieprijzen

Energie	Prijs	Bron
Benzine (Euro95 / E10)	€ 2,42 per liter	Gemiddelde prijs volgens de ANWB (anwb.nl), oktober 2026
Diesel (B7)	€ 2,50 per liter	Gemiddelde prijs volgens de ANWB (anwb.nl), oktober 2026
LPG	€ 0,99 per liter	Mediaan van alle pompen (hoeveelbenzine.nl), 5 oktober 2026
Elektriciteit	€ 0,34 per kWh	Mix van 70% thuis laden (€ 0,28) en 30% publiek laden (€ 0,49) (joulo.nl), 2026
Waterstof	€ 21,00 per kg	Gemiddelde pompprijs; prijzen liggen tussen € 16,95 en € 25 per kg (anwb.nl, veel.nl), 2026

Keuze bij elektriciteit: de prijs per kWh hangt sterk af van waar je laadt. Ik ga uit van een mix van vooral thuis laden en soms publiek laden. Snelladen laat ik buiten beschouwing.

Voorbeeldverbruik

Het verbruik vult de eigenaar per auto in. Voor de testdata en de voorbeelden gebruik ik deze gangbare waarden:

Soort	Voorbeeldverbruik	Kosten per km	Bron verbruik
ICE benzine	6 liter / 100 km	€ 0,145	eigen aanname, gangbaar voor een middenklasser
ICE diesel	5 liter / 100 km	€ 0,125	eigen aanname
ICE LPG	8 liter / 100 km	€ 0,079	eigen aanname
BEV	17 kWh / 100 km	€ 0,058	joulo.nl
FCEV	1 kg / 100 km	€ 0,210	anwb.nl

TCO-bedragen per jaar

Uitgangspunt: een middenklasse auto en 15.000 km per jaar.

Onderdeel	ICE	BEV	FCEV	Bron / toelichting
Afschrijving	15% van aanschafprijs	15% van aanschafprijs	15% van aanschafprijs	Vereenvoudiging: een vast percentage per jaar
Verzekering	€ 800	€ 1.100	€ 1.100	Schatting. Een elektrische auto is € 240 – € 360 per jaar duurder om te verzekeren (anwb.nl). Voor FCEV neem ik hetzelfde aan als voor BEV.
Wegenbelasting	€ 1.000	€ 710	€ 710	Middenklasser: benzine ca. € 83 per maand, elektrisch ca. € 59 per maand. Elektrische en waterstofauto's krijgen in 2026 30% korting (evtrader.com).
Onderhoud	€ 650	€ 385	€ 600	Gemiddelden voor een middenklasser (echteautokosten.nl). Voor FCEV geen betrouwbare bron gevonden; eigen schatting.
Energie	15.000 × kosten per km	15.000 × kosten per km	15.000 × kosten per km	Zie energieprijzen

Vereenvoudigingen:

- Voor ICE gebruik ik één bedrag voor de wegenbelasting. In werkelijkheid betalen diesel- en LPG-auto's een toeslag en hangt het bedrag af van het gewicht en de provincie.
- Afschrijving is in werkelijkheid in de eerste jaren hoger dan later. Een vast percentage is eenvoudiger en voor deze casus voldoende.

Rekenvoorbeelden

Benzineauto van € 30.000, verbruik 6 liter / 100 km:

Onderdeel	Bedrag
Afschrijving (15% × € 30.000)	€ 4.500
Verzekering	€ 800
Wegenbelasting	€ 1.000
Onderhoud	€ 650
Energie (15.000 × € 0,1452)	€ 2.178
TCO per jaar	**€ 9.128** (≈ € 0,61 per km)

Elektrische auto (BEV) van € 35.000, verbruik 17 kWh / 100 km:

Onderdeel	Bedrag
Afschrijving (15% × € 35.000)	€ 5.250
Verzekering	€ 1.100
Wegenbelasting	€ 710
Onderhoud	€ 385
Energie (15.000 × € 0,0578)	€ 867
TCO per jaar	**€ 8.312** (≈ € 0,55 per km)

Huurprijs: een auto met een uurtarief van € 6,00 en een km-tarief van € 0,25, gehuurd voor 4 uur met 80 verwachte kilometers:

> 4 × € 6,00 + 80 × € 0,25 = € 24,00 + € 20,00 = **€ 44,00**

Deze rekenvoorbeelden worden ook gebruikt als testgevallen voor de unit tests (hoofdstuk 4).